KOLBEN ERP

Documentación técnica de infraestructura y desarrollo

Versión 1.2 · 30 de octubre de 2026

Desarrollo: Diego Aguilar · Cliente: Javier Escalante (Kolben)

Documento vivo: se ajusta conforme avance el proyecto. Las funciones, fórmulas, casos numéricos y pruebas de entrega del sistema están en la Especificación funcional v1.1 (28/10/2026); este documento cubre cómo se construye, se modela en base de datos, se prueba y se publica.

1. Resumen y entornos

Kolben ERP es un sistema web administrativo interno (ERP a la medida, un solo usuario con control total) para Kolben, distribuidora de repuestos (cilindros de freno y clutch). Reemplaza el control actual en hojas sueltas y cubre el ciclo completo: productos y precios por cliente, proformas y facturas, cuentas por cobrar, inventario por lotes (FIFO: *por lote, primero el más viejo*), pedidos al proveedor y costeo de importación, utilidad, caja y resumen ejecutivo.

Relación con CatalogoHN

- **Proyectos 100% separados:** CatalogoHN (`catalogohn.com`, SaaS multi-inquilino de catálogos, puerto 3001) y Kolben ERP tienen repositorio, código, modelos y base de datos independientes.
- **Integración futura (opcional):** el ERP podría conectarse por API REST/Webhooks con CatalogoHN (por ejemplo, descontar stock público al facturar o recibir pedidos como proformas). Hoy no existe; los pedidos de CatalogoHN se ingresan a mano en el POS. El backend se diseña con una API limpia bajo `/api` para dejar esa puerta abierta.

Entornos

Ambos proyectos conviven en el mismo Droplet de DigitalOcean (Ubuntu LTS), con Nginx como proxy inverso y certificados gratuitos de Let's Encrypt. Kolben ERP tiene dos entornos completos: **pruebas** (rama `develop`) y **producción** (rama `main`), cada uno con puerto, carpeta, proceso PM2 y base de datos propios, dentro de la misma instancia de PostgreSQL pero aisladas de la base de CatalogoHN.

	Pruebas (test)	Producción
Rama de Git	`develop`	`main`
Dominio	test.app.kolben.catalogohn.com	app.kolben.catalogohn.com
Puerto API (interno)	4001	4002
Base de datos	`kolben_erp_test`	`kolben_erp`
Carpeta en el Droplet	`/var/www/kolben-erp-test`	`/var/www/kolben-erp`
Proceso PM2	`kolben-api-test`	`kolben-api-prod`
Archivos subidos	`/var/www/kolben-erp-test/backend/uploads/`	`/var/www/kolben-erp/backend/uploads/`
Frontend (estático)	`.../frontend/dist` servido por Nginx	`.../frontend/dist` servido por Nginx

Nota: Falta crear en el DNS un registro A por cada subdominio apuntando a la IP del Droplet. Cuando exista un nombre más corto, solo cambia `server_name`, `APP_URL` y el certificado.

Cómo se conectan las piezas

- El navegador entra por HTTPS a Nginx (puertos 80/443). Nginx entrega el frontend compilado (React) y envía todo lo que empiece con `/api/` al backend en `127.0.0.1:4001` o `:4002`.
- Como frontend y API comparten el mismo dominio, la cookie HttpOnly del JWT funciona sin CORS ni configuración especial.
- Los puertos 4001 y 4002 **no se abren al exterior**: el backend escucha solo en `127.0.0.1`. CatalogoHN usa el 3001; no hay conflicto.
- PostgreSQL escucha solo en `localhost`. Pruebas y producción usan bases y usuarios distintos, y ninguno tiene permiso sobre la base de CatalogoHN.

2. Stack tecnológico

Capa	Tecnología	Notas
Frontend	React + Vite + TypeScript	SPA con diseño único de 1280 px escalado proporcionalmente al ancho del aparato (laptop, tablet, celular) sin desplazamiento horizontal.
UI	Tailwind CSS + shadcn/ui	Tipografía Inter con números tabulares (`tabular-nums`). Confirmación en línea (no modal).
Tablas	`@tanstack/react-table`	Ventas, Productos, Clientes, flujo de Caja.
Backend	Node.js + Express + TypeScript	API REST bajo `/api`.
Base de datos	PostgreSQL local en el Droplet	Una base por entorno.
ORM	Prisma	Migraciones versionadas en el repositorio.
Autenticación	JWT en cookie HttpOnly	Contraseña con hash (bcrypt o argon2). Cookie `Secure` + `SameSite=Lax`.
Archivos	Disco local del Droplet	Nginx sirve vía endpoint protegido (sección 7).
PDFs	`@react-pdf/renderer`	Proforma, factura, catálogo, cotización, respaldo de Por cobrar. Se generan en el backend con `renderToBuffer` para poder enviarlos por correo.
Moneda y formato	`decimal.js` + `Intl.NumberFormat("es-HN")`	Confirmado. Columnas `NUMERIC` en PostgreSQL; nunca `float`. Lempiras con 2 decimales en pantalla; porcentajes con 1 decimal.
Excel	`exceljs`	Confirmado. La spec lo pide: cotización de la Calculadora y descargable de pedido.

Capa	Tecnología	Notas
Imágenes	`sharp`	Redimensiona fotos y logos al subirlas: menos peso en disco y catálogo PDF más rápido.
Validación	`zod`	Mismo esquema para validar entradas del API.
Procesos programados	`node-cron`	Cierre de mes (día 1 a las 00:00 hora de Honduras `UTC-6`) y respaldo semanal por correo. PM2 en modo fork con 1 instancia.
Correo	`nodemailer`	Respaldo semanal de Por cobrar (PDF).
Seguridad	`helmet`, `express-rate-limit`	Límite de intentos en el login.
Procesos / Web	PM2 + Nginx + Certbot	HTTPS con Let's Encrypt.

3. Estructura del repositorio (monorepo)

Un solo repositorio de GitHub con dos aplicaciones. Cada entorno del Droplet es un clon del mismo repositorio, parado en su rama.

```
kolben-erp/
├── backend/
│   ├── prisma/
│   │   ├── schema.prisma
│   │   ├── seed.ts                # datos de prueba (solo local y pruebas)
│   │   └── migrations/
│   ├── src/
│   │   ├── server.ts              # arranque (escucha en 127.0.0.1)
│   │   ├── app.ts                 # express, helmet, rutas
│   │   ├── config/env.ts          # lee y valida .env con zod
│   │   ├── middlewares/           # auth (JWT cookie), errores, rate limit
│   │   ├── modules/
│   │   │   ├── auth/
│   │   │   ├── config/            # proforma, bancos autorizados, ISV, respaldo
│   │   │   ├── productos/         # productos, categorias, marcas, lotes, ajustes
│   │   │   ├── clientes/          # clientes, zonas, ciudades, precios
│   │   │   ├── pos/               # proformas y apartado
│   │   │   ├── ventas/            # facturas, abonos, cambios
│   │   │   ├── compras/           # calculadora, pedidos, pagos, recepcion
│   │   │   ├── caja/              # saldos, cuentas banco, gastos, flujo, cierre anio
│   │   │   ├── reportes/          # resumen, ventas/utilidad por codigo
│   │   │   └── catalogo/
│   │   ├── services/              # fifo, costeo, multiplicador, pe-meta, mes-sin-stock
│   │   ├── pdf/                   # plantillas react-pdf
│   │   ├── jobs/                  # cierre-mes, respaldo-semanal
│   │   └── lib/                   # prisma client, decimal, historial
│   ├── uploads/                   # NO se sube a Git
│   │   └── facturas/ logos/ comprobantes/ productos/ catalogo/
│   ├── .env                       # NO se sube a Git
│   ├── package.json
│   └── tsconfig.json
├── frontend/
```

```
│   ├── src/
│   │   ├── pages/                 # Resumen, POS, Ventas, Clientes, ...
│   │   ├── components/ui/         # componentes base
│   │   ├── components/            # tablas, formularios, layout 1280px
│   │   ├── lib/                   # api (fetch), formato de moneda y fechas
│   │   └── main.tsx
│   ├── .env.production            # VITE_API_URL=/api
│   ├── vite.config.ts
│   └── package.json
├── scripts/
│   ├── deploy.sh
│   ├── backup.sh
│   ├── caja_backup.sh
│   └── reset-password.ts          # restablece la contraseña (se corre en el servidor)
├── ecosystem.config.js            # PM2
├── docs/                          # especificacion v1.1 y este documento
├── GEMINI.md                      # reglas y contexto permanente del proyecto
├── .gitignore
└── README.md
```

.gitignore mínimo: `node_modules/`, `dist/`, `.env`, `.env.*` (excepto los `.example`), `backend/uploads/*` y `*.dump`.

4. Variables de entorno

Cada entorno tiene su propio `backend/.env` en el servidor (nunca en Git). En el repositorio solo va `backend/.env.example` con valores de ejemplo.

backend/.env (producción)

```
NODE_ENV=production
HOST=127.0.0.1
PORT=4002
TZ=America/Tegucigalpa

DATABASE_URL="postgresql://kolben_prod:CAMBIAR_PASSWORD@localhost:5432/kolben_erp?schema=public"

JWT_SECRET=CAMBIAR_por_cadena_larga_aleatoria_64_caracteres
JWT_EXPIRES_IN=8h
COOKIE_NAME=kolben_token
COOKIE_SECURE=true

APP_URL=https://app.kolben.catalogohn.com
UPLOADS_DIR=/var/www/kolben-erp/backend/uploads

SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=correo@ejemplo.com
SMTP_PASS=CAMBIAR_app_password
SMTP_FROM="Kolben ERP <correo@ejemplo.com>"
```

Qué cambia en pruebas

Variable	Pruebas	Local (tu computadora)
`NODE_ENV`	`test`	`development`
`PORT`	`4001`	`4000`
`DATABASE_URL`	`.../kolben_erp_test` con usuario `kolben_test`	`.../kolben_erp_local`
`COOKIE_SECURE`	`true`	`false` (sin HTTPS)
`APP_URL`	https://test.app.kolben.catalogohn.com	http://localhost:5173
`UPLOADS_DIR`	`/var/www/kolben-erp-test/backend/uploads`	`./uploads`
`JWT_SECRET`	Distinto al de producción	Cualquiera

En pruebas conviene apuntar el correo a tu propio correo para no enviar el respaldo semanal a Javier por error.

frontend/.env.production

```
VITE_API_URL=/api
```

Al ser ruta relativa, el mismo build funciona en pruebas y producción. En local, `vite.config.ts` define un proxy de `/api` hacia `http://localhost:4000`.

5. Flujo de trabajo con Git

Todo cambio se desarrolla en tu computadora, se sube a `develop`, se prueba en el entorno de pruebas del Droplet y, solo si está bien, se pasa a `main` y a producción.

```
tu PC ──push──▶ GitHub: develop ──git pull──▶ Droplet: PRUEBAS
                      │ (si todo está bien)
                    merge
                      ▼
                GitHub: main    ──git pull──▶ Droplet: PRODUCCIÓN
```

Paso 1 · Preparar el repositorio (una sola vez)

```
git init
git remote add origin git@github.com:TU_USUARIO/kolben-erp.git
git checkout -b main
git add .
git commit -m "chore: estructura inicial"
git push -u origin main
git checkout -b develop
git push -u origin develop
```

Recomendado: en GitHub, proteger `main` (Settings → Branches) para no subir directo por error.

Paso 2 · Trabajar y subir a develop (día a día)

```
git checkout develop
git pull origin develop
# ... programar ...
git add .
git commit -m "feat(pos): guardar proforma y apartar unidades"
git push origin develop
```

Prefijos de commit sugeridos: `feat`, `fix`, `chore`, `docs`, `refactor`. Si un cambio es grande, crea una rama `feature/nombre` desde `develop`, y luego haz merge a `develop`.

Paso 3 · Desplegar en PRUEBAS (Droplet)

```
ssh usuario@IP_DEL_DROPLET
cd /var/www/kolben-erp-test
git pull origin develop

cd backend
npm ci
npx prisma migrate deploy
npx prisma generate
npm run build

cd ../frontend
npm ci
npm run build

pm2 restart kolben-api-test
pm2 logs kolben-api-test --lines 50
```

Prueba en https://test.app.kolben.catalogohn.com. Si algo falla, corriges en tu PC y repites desde el Paso 2.

Paso 4 · Pasar a main (cuando pruebas está bien)

```
git checkout main
git pull origin main
git merge develop
git push origin main
git tag -a v1.0.0 -m "Primera versión estable"
git push origin v1.0.0
git checkout develop
```

Paso 5 · Desplegar en PRODUCCIÓN (Droplet)

Antes de tocar producción, haz siempre un respaldo (el script está en la sección 8).

```
ssh usuario@IP_DEL_DROPLET
/var/www/kolben-erp/scripts/backup.sh

cd /var/www/kolben-erp
```

```
git pull origin main

cd backend
npm ci
npx prisma migrate deploy
npx prisma generate
npm run build

cd ../frontend
npm ci
npm run build

pm2 restart kolben-api-prod
pm2 logs kolben-api-prod --lines 50
```

Script opcional: deploy.sh

Hace los mismos pasos con un solo comando (`./scripts/deploy.sh test` o `./scripts/deploy.sh prod`). Sigue siendo manual: lo ejecutas tú.

```
#!/bin/bash
set -e
ENV=$1
if [ "$ENV" = "test" ]; then
  DIR=/var/www/kolben-erp-test; BRANCH=develop; APP=kolben-api-test
elif [ "$ENV" = "prod" ]; then
  DIR=/var/www/kolben-erp; BRANCH=main; APP=kolben-api-prod
  /var/www/kolben-erp/scripts/backup.sh
else
  echo "Uso: ./scripts/deploy.sh test|prod"; exit 1
fi

cd $DIR
git pull origin $BRANCH
cd backend && npm ci && npx prisma migrate deploy && npx prisma generate
npm run build
cd ../frontend && npm ci && npm run build
pm2 restart $APP
echo "Listo: $APP"
```

Migraciones de base de datos

- En tu PC: `npx prisma migrate dev --name descripcion_del_cambio` crea y aplica la migración. La carpeta `prisma/migrations` **sí se sube a Git**.
- En el Droplet: solo `npx prisma migrate deploy` (aplica las pendientes, nunca borra datos).
- Nunca uses `migrate dev` ni `migrate reset` en producción.

Volver a una versión anterior

```
cd /var/www/kolben-erp
git fetch --tags
git checkout v1.0.0
```

```
# repetir build de backend y frontend, luego:
pm2 restart kolben-api-prod
# volver a seguir main después:
git checkout main
```

Si la versión anterior tenía otra estructura de base de datos, restaura primero el respaldo (sección 8).

6. Preparación del Droplet (una sola vez)

Paso 1 · Paquetes base

```
sudo apt update && sudo apt install -y nginx postgresql git certbot python3-certbot-nginx rclone
# Node.js 20 LTS (o 22 LTS); usar la misma versión que ya corre CatalogoHN
sudo npm install -g pm2
```

Paso 2 · Bases de datos y usuarios separados

```
sudo -u postgres psql
CREATE USER kolben_prod WITH PASSWORD 'CAMBIAR_1';
CREATE USER kolben_test WITH PASSWORD 'CAMBIAR_2';
CREATE DATABASE kolben_erp OWNER kolben_prod;
CREATE DATABASE kolben_erp_test OWNER kolben_test;
q
```

Paso 3 · Clonar el repositorio dos veces

```
cd /var/www
git clone -b main    git@github.com:TU_USUARIO/kolben-erp.git kolben-erp
git clone -b develop git@github.com:TU_USUARIO/kolben-erp.git kolben-erp-test

mkdir -p kolben-erp/backend/uploads/{facturas,logos,comprobantes,productos,catalogo}
mkdir -p kolben-erp-test/backend/uploads/{facturas,logos,comprobantes,productos,catalogo}
# crear backend/.env en cada carpeta (sección 4), luego primer despliegue
```

Paso 4 · Firewall

```
sudo ufw allow OpenSSH
sudo ufw allow 'Nginx Full'
sudo ufw enable
# NO abrir 4001 ni 4002
```

Si el Droplet ya tiene reglas de firewall por CatalogoHN, solo verifica que 4001 y 4002 no estén abiertos.

Paso 5 · PM2 (ecosystem.config.js en la raíz del repo)

```
module.exports = {
  apps: [
    {
      name: 'kolben-api-test',
      cwd: '/var/www/kolben-erp-test/backend',
```

```
      script: 'dist/server.js',
      instances: 1,
      exec_mode: 'fork',
      max_memory_restart: '512M',
    },
    {
      name: 'kolben-api-prod',
      cwd: '/var/www/kolben-erp/backend',
      script: 'dist/server.js',
      instances: 1,
      exec_mode: 'fork',
      max_memory_restart: '512M',
    },
  ],
};
```

```
pm2 start /var/www/kolben-erp/ecosystem.config.js
pm2 save
pm2 startup   # ejecutar el comando que muestre
pm2 status
```

Cada proceso lee su propio `.env` (PORT, base de datos, etc.) desde su carpeta. El límite de 512 MB por proceso deja margen para generar PDFs con fotos (catálogo); con el Droplet en torno al 45% de RAM hay espacio, pero conviene vigilarlo con `pm2 monit` la primera vez que se genere el catálogo. Debe ser **una sola instancia**, porque los procesos programados (cierre de mes, respaldo semanal) no deben ejecutarse duplicados.

Paso 6 · Nginx y HTTPS

Archivo `/etc/nginx/sites-available/kolben-erp` (producción):

```
server {
    listen 80;
    server_name app.kolben.catalogohn.com;

    root /var/www/kolben-erp/frontend/dist;
    index index.html;
    client_max_body_size 10M;

    location / {
        try_files $uri /index.html;
    }

    location /api/ {
        proxy_pass http://127.0.0.1:4002;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # Archivos privados: solo se entregan si el backend lo autoriza
    location /protected-uploads/ {
        internal;
        alias /var/www/kolben-erp/backend/uploads/;
```

```
    }
}
```

Archivo `/etc/nginx/sites-available/kolben-erp-test` (pruebas): igual, cambiando `server_name test.app.kolben.catalogohn.com`, el `root` a `/var/www/kolben-erp-test/frontend/dist`, el puerto a `4001` y el `alias` a `/var/www/kolben-erp-test/backend/uploads/`. Se recomienda agregar `add_header X-Robots-Tag "noindex";`.

```
sudo ln -s /etc/nginx/sites-available/kolben-erp /etc/nginx/sites-enabled/
sudo ln -s /etc/nginx/sites-available/kolben-erp-test /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
sudo certbot --nginx -d app.kolben.catalogohn.com -d test.app.kolben.catalogohn.com
```

Certbot agrega el HTTPS y la redirección automáticamente, y renueva el certificado solo.

7. Archivos subidos (facturas, logos, comprobantes)

Todo se guarda en `backend/uploads/` del entorno correspondiente, ordenado por tipo: `facturas/`, `logos/` (marcas), `comprobantes/`, `productos/` (fotos) y `catalogo/` (portada).

- **Recomendado: endpoint protegido.** El navegador pide `/api/files/:tipo/:nombre`; el backend valida la cookie JWT y responde con la cabecera `X-Accel-Redirect: /protected-uploads/tipo/nombre`. Nginx entrega el archivo directamente, sin pasar el contenido por Node. Sin sesión, no hay acceso.
- Alternativa (solo para archivos no sensibles como logos o fotos de productos): `location /uploads/` estático. No recomendado para facturas ni comprobantes.
- En la base de datos se guarda solo la ruta relativa (por ejemplo `facturas/1504.pdf`), nunca la ruta absoluta.
- Subidas con `multer`: validar tipo (PDF, JPG, PNG, WebP) y tamaño máximo; renombrar el archivo con un identificador propio.
- La carpeta `uploads/` entra en el respaldo diario (sección 8) y no se versiona en Git.

8. Respaldos automáticos

La spec exige una copia diaria de la base de datos **fuera del servidor**. Se propone lo siguiente para producción (en pruebas no es necesario):

Qué	Cuándo	Retención	Contenido
Base de datos completa	Diario 2:00 a. m.	30 días en el Droplet; copia externa	`pg_dump` formato custom de `kolben_erp`
Carpeta `uploads/`	Diario 2:00 a. m.	30 días	Archivo `.tar.gz`

Qué	Cuándo	Retención	Contenido
Datos de Caja (CSV)	Semanal, lunes 2:30 a. m.	12 semanas	Saldos, cuentas de banco, gastos fijos y variables
PDF de Por cobrar	Semanal, día configurado	En el correo	Lo envía el sistema (spec, sección 7)

Configurar rclone con Google Drive (una sola vez)

El Droplet no tiene navegador, así que la autorización de Google se hace desde tu computadora.

```
# 1) En el Droplet
sudo apt install -y rclone
rclone config
# n) New remote → nombre: kolben-remoto → tipo: drive
# scope: drive.file (solo ve lo que rclone crea)
# "Use web browser to automatically authenticate?" → n
# copia el comando "rclone authorize" que muestra

# 2) En tu computadora (con rclone instalado)
rclone authorize "drive" "TOKEN_QUE_MOSTRO_EL_DROPLET"
# se abre el navegador, inicia sesión y acepta; copia el token que imprime

# 3) De vuelta en el Droplet: pega el token y termina la configuración
rclone lsd kolben-remoto:   # prueba de conexión
rclone mkdir kolben-remoto:kolben-backups
```

El token queda guardado en `~/.config/rclone/rclone.conf` del usuario que corre el cron: protégelo (`chmod 600`) y no lo subas a Git. Usar una cuenta de Google propia del negocio, no una personal compartida.

Credenciales sin contraseña en el script

```
# archivo ~/.pgpass del usuario que corre el cron (permisos 600)
localhost:5432:kolben_erp:kolben_prod:CAMBIAR_1
chmod 600 ~/.pgpass
```

scripts/backup.sh

```
#!/bin/bash
set -e
# si cualquier paso falla, avisa a Healthchecks.io (que envía correo)
trap 'curl -fsS -m 10 https://hc-ping.com/TU_UUID_BACKUP/fail > /dev/null' ERR
FECHA=$(date +%Y-%m-%d_%H%M)
DEST=/var/backups/kolben
mkdir -p $DEST

pg_dump -U kolben_prod -h localhost -Fc kolben_erp > $DEST/kolben_erp_$FECHA.dump
tar -czf $DEST/uploads_$FECHA.tar.gz -C /var/www/kolben-erp/backend uploads
```

```
# borrar copias locales de más de 30 días
find $DEST -type f -mtime +30 -delete

# copia fuera del servidor (Google Drive, remoto configurado como 'kolben-remoto')
rclone copy $DEST kolben-remoto:kolben-backups --max-age 2d
# aviso de éxito: si el ping no llega a tiempo, Healthchecks.io también avisa
curl -fsS -m 10 --retry 3 https://hc-ping.com/TU_UUID_BACKUP > /dev/null
echo "Respaldo OK $FECHA"
```

scripts/caja_backup.sh

```
#!/bin/bash
set -e
FECHA=$(date +%Y-%m-%d)
DEST=/var/backups/kolben/caja
mkdir -p $DEST
for T in SaldoCaja CuentaBanco Banco MontoFijoVigente GastoFijoMes MovimientoGasto CategoriaGasto
Anio MesContable; do
  psql -U kolben_prod -h localhost -d kolben_erp     -c "copy (SELECT * FROM "$T") TO
'$DEST/$T-$FECHA.csv' CSV HEADER"
done
find $DEST -type f -mtime +84 -delete
rclone copy $DEST kolben-remoto:kolben-backups/caja --max-age 8d
```

Cron (sudo crontab -e)

```
0 2 * * *  /var/www/kolben-erp/scripts/backup.sh      >> /var/log/kolben-backup.log 2>&1
30 2 * * 1 /var/www/kolben-erp/scripts/caja_backup.sh >> /var/log/kolben-backup.log 2>&1
```

Avisos y mantenimiento

- **Aviso si falla el respaldo:** cuenta gratuita en Healthchecks.io. El script le hace un ping al terminar bien y otro a `/fail` si algo sale mal; si el ping no llega a la hora esperada, Healthchecks envía un correo. Usar un chequeo distinto para `caja_backup.sh`.
- **Rotación de logs:** que `/var/log/kolben-backup.log` y los logs de PM2 no crezcan sin límite (`pm2 install pm2-logrotate` ; para el log de respaldo, un archivo en `/etc/logrotate.d/`).
- **Espacio en disco:** revisar de vez en cuando con `df -h` el crecimiento de `uploads/` y `/var/backups/kolben` .

Restaurar un respaldo

```
pg_restore -U kolben_prod -h localhost -d kolben_erp --clean --if-exists
/var/backups/kolben/kolben_erp_AAAA-MM-DD_HHMM.dump
tar -xzf uploads_AAAA-MM-DD_HHMM.tar.gz -C /var/www/kolben-erp/backend
```

Probar la restauración en la base de pruebas al menos una vez antes del arranque. Un respaldo que nunca se ha restaurado no está comprobado.

9. Modelo de datos (Prisma — Especificación v1.1)

El diagrama de relaciones está en el **Anexo B**. Actualizado conforme a la sección 2 de la **Especificación funcional v1.1 (28/10/2026)**. Los montos usan `Decimal`; los cálculos internos van con más decimales que los 2 que se muestran en pantalla.

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

enum TipoVenta {
  CREDITO
  CONTADO
}

enum EstadoProforma {
  PENDIENTE
  INGRESADA
  ELIMINADA
}

enum TipoLinea {
  NORMAL
  CAMBIO
}

enum EstadoPedido {
  EN_PRODUCCION
  EN_CAMINO
  RECIBIDO
}

enum TipoPagoPedido {
  PROVEEDOR
  GASTOS
}

enum MedioAbono {
  EFECTIVO
  TRANSFERENCIA
  CHEQUE
  DEVOLUCION
  CARGA_INICIAL
}

enum GrupoGasto {
  NEGOCIO
  PERSONAL
}

enum TipoGasto {
  FIJO
```

```
  VARIABLE
}

// ---------- Acceso y configuración ----------
model Usuario {
  id           Int      @id @default(autoincrement())
  usuario      String   @unique
  passwordHash String
  creadoEn     DateTime @default(now())
}

model Configuracion { // registro único (id = 1)
  id             Int      @id @default(1)
  vendedor       String
  contacto       String
  titularCheques String?
  porcentajeIsv  Decimal  @default(15) @db.Decimal(5, 2)
  correoRespaldo String?
  diaRespaldo    Int      @default(1)
  portadaPath    String?  // portada del catálogo PDF
  paletaCatalogo String   @default("SISTEMA_KOLBEN")
}

model Banco { // bancos autorizados de la proforma
  id                Int    @id @default(autoincrement())
  nombre            String
  cuentaEnmascarada String
  orden             Int
}

// ---------- Listas ----------
model Categoria {
  id            Int        @id @default(autoincrement())
  nombre        String     @unique
  orden         Int
  empaque       Int        @default(10) // 10 / 20 / 50
  multiplicador Decimal    @default(2.0) @db.Decimal(6, 3)
  productos     Producto[]
}

model Marca {
  id        Int        @id @default(autoincrement())
  nombre    String     @unique
  logoPath  String?
  orden     Int
  productos Producto[]
}

model Zona {
  id       Int      @id @default(autoincrement())
  nombre   String   @unique
  orden    Int
  ciudades Ciudad[]
}

model Ciudad {
  id       Int       @id @default(autoincrement())
  nombre   String
  orden    Int
```

```
  zonaId   Int
  zona     Zona      @relation(fields: [zonaId], references: [id])
  clientes Cliente[]
}

// ---------- Productos y stock ----------
model Producto {
  id                  Int                  @id @default(autoincrement())
  codigo              String               @unique
  categoriaId         Int
  marcaId             Int
  descripcion         String               // una línea por dato
  cbmUnidad           Decimal              @default(0) @db.Decimal(8, 4)
  fotoPath            String?
  activo              Boolean              @default(true)
  posicion            Int
  categoria           Categoria            @relation(fields: [categoriaId], references: [id])
  marca               Marca                @relation(fields: [marcaId], references: [id])
  lotes               Lote[]
  ajustes             AjusteStock[]
  precios             PrecioCliente[]
  lineasProforma      LineaProforma[]
  lineasFactura       LineaFactura[]
  lineasPedido        LineaPedido[]
  ventasBase          VentaBase[]
  mesesSinStock       MesSinStock[]
  periodosSinStock    PeriodoSinStock[]
  borradorCalculadora BorradorCalculadora?

  @@index([categoriaId, marcaId, posicion])
}

model Lote { // FIFO: se descuenta del más viejo con quedan > 0
  id            Int           @id @default(autoincrement())
  productoId    Int
  pedidoId      Int?          // null = lote Inicial
  esInicial     Boolean       @default(false)
  fechaRecibido DateTime
  cantidad      Int
  quedan        Int
  fobUnidad     Decimal       @db.Decimal(14, 4)
  aduanaUnidad  Decimal       @db.Decimal(14, 4)
  costoUnidad   Decimal       @db.Decimal(14, 4)
  producto      Producto      @relation(fields: [productoId], references: [id])
  pedido        Pedido?       @relation(fields: [pedidoId], references: [id])
  consumos      LoteConsumo[]

  @@index([productoId, fechaRecibido])
}

model AjusteStock {
  id           Int      @id @default(autoincrement())
  productoId   Int
  fecha        DateTime @default(now())
  unidades     Int      // positivo entra (al más nuevo), negativo sale (del más viejo)
  motivo       String
  stockAntes   Int
  stockDespues Int
  producto     Producto @relation(fields: [productoId], references: [id])
```

```
}

// ---------- Clientes y precios ----------
model Cliente {
  id          Int             @id @default(autoincrement())
  codigo      String          @unique
  nombre      String
  ciudadId    Int
  direccion   String?
  contacto    String?
  diasCredito Int             @default(0)
  aplicaIsv   Boolean         @default(false)
  activo      Boolean         @default(true)
  ciudad      Ciudad          @relation(fields: [ciudadId], references: [id])
  precios     PrecioCliente[]
  proformas   Proforma[]
  facturas    Factura[]
}

model PrecioCliente { // precio sin ISV; cambios van a Historial
  id         Int      @id @default(autoincrement())
  clienteId  Int
  productoId Int
  precio     Decimal  @db.Decimal(14, 2)
  fecha      DateTime @default(now())
  cliente    Cliente  @relation(fields: [clienteId], references: [id])
  producto   Producto @relation(fields: [productoId], references: [id])

  @@unique([clienteId, productoId])
}

// ---------- Proformas, facturas y cobros ----------
model Proforma {
  id        Int             @id @default(autoincrement())
  numero    Int             @unique
  clienteId Int
  fecha     DateTime
  dias      Int
  vence     DateTime
  tipo      TipoVenta       @default(CREDITO)
  aplicaIsv Boolean         @default(false)
  estado    EstadoProforma  @default(PENDIENTE)
  creadoEn  DateTime        @default(now())
  cliente   Cliente         @relation(fields: [clienteId], references: [id])
  lineas    LineaProforma[]
  factura   Factura?
}

model LineaProforma {
  id             Int       @id @default(autoincrement())
  proformaId     Int
  productoId     Int
  cantidad       Int
  precio         Decimal   @db.Decimal(14, 2)
  precioCambiado Boolean   @default(false)
  tipo           TipoLinea @default(NORMAL)
  proforma       Proforma  @relation(fields: [proformaId], references: [id], onDelete: Cascade)
  producto       Producto  @relation(fields: [productoId], references: [id])
}
```

```
model Factura {
  id           Int            @id @default(autoincrement())
  numero       Int            @unique // interno, no es factura SAR
  proformaId   Int?           @unique
  clienteId    Int
  fecha        DateTime
  vence        DateTime
  tipo         TipoVenta
  subtotal     Decimal        @db.Decimal(14, 2)
  isv          Decimal        @default(0) @db.Decimal(14, 2)
  total        Decimal        @db.Decimal(14, 2)
  cargaInicial Boolean        @default(false)
  eliminada    Boolean        @default(false)
  eliminadaEn  DateTime?
  proforma     Proforma?      @relation(fields: [proformaId], references: [id])
  cliente      Cliente        @relation(fields: [clienteId], references: [id])
  lineas       LineaFactura[]
  abonos       Abono[]

  @@index([clienteId, fecha])
  @@index([fecha])
}

model LineaFactura {
  id               Int           @id @default(autoincrement())
  facturaId        Int
  productoId       Int
  descripcion      String        // texto conservado al facturar
  cantidadOriginal Int
  cantidad         Int           // baja si se edita la factura
  precio           Decimal       @db.Decimal(14, 2)
  tipo             TipoLinea     @default(NORMAL)
  costoFifo        Decimal       @default(0) @db.Decimal(14, 4)
  factura          Factura       @relation(fields: [facturaId], references: [id], onDelete:
Cascade)
  producto         Producto      @relation(fields: [productoId], references: [id])
  consumos         LoteConsumo[]
}

model LoteConsumo { // Salida por lote: de qué lote salió cada pieza (costo, multiplicador y
devolución)
  id             Int          @id @default(autoincrement())
  lineaFacturaId Int
  loteId         Int
  cantidad       Int
  costoUnidad    Decimal      @db.Decimal(14, 4)
  fobUnidad      Decimal      @default(0) @db.Decimal(14, 4)
  lineaFactura   LineaFactura @relation(fields: [lineaFacturaId], references: [id], onDelete:
Cascade)
  lote           Lote         @relation(fields: [loteId], references: [id])
}

model Abono {
  id                  Int        @id @default(autoincrement())
  facturaId           Int
  fecha               DateTime
  monto               Decimal    @db.Decimal(14, 2)
  numeroRecibo        String?
```

```
  medio                MedioAbono
  chequePendienteHasta DateTime?
  factura              Factura    @relation(fields: [facturaId], references: [id])

  @@index([facturaId])
}

// ---------- Compras y Calculadora ----------
model ParametrosCompra { // registro único (id = 1) de la Calculadora
  id                   Int     @id @default(1)
  mesesCiclo           Int[]   @default([3, 9])
  porcentajeStockExtra Decimal @default(15) @db.Decimal(5, 2)
}

model BorradorCalculadora { // borrador hasta ingresar la orden
  id           Int      @id @default(autoincrement())
  productoId   Int      @unique
  ajuste       Int      @default(0)
  fobUnidad    Decimal  @db.Decimal(14, 4)
  escritoAMano Boolean  @default(false)
  producto     Producto @relation(fields: [productoId], references: [id], onDelete: Cascade)
}

model Pedido {
  id               Int              @id @default(autoincrement())
  numero           Int              @unique
  estado           EstadoPedido     @default(EN_PRODUCCION)
  fechaColocado    DateTime
  llegadaAprox     DateTime?
  porcentajeAduana Decimal          @db.Decimal(5, 2)
  fobEstimado      Decimal          @db.Decimal(14, 2)
  fobReal          Decimal?         @db.Decimal(14, 2)
  proveedorPagado  Boolean          @default(false)
  cbmPackingList   Decimal?         @db.Decimal(10, 4)
  recibidoEn       DateTime?
  lineas           LineaPedido[]
  pagos            PagoPedido[]
  gastos           GastoRecepcion[]
  lotes            Lote[]
}

model LineaPedido {
  id         Int      @id @default(autoincrement())
  pedidoId   Int
  productoId Int
  cantidad   Int
  fobUnidad  Decimal  @db.Decimal(14, 4)
  cbmUnidad  Decimal  @db.Decimal(8, 4)
  pedido     Pedido   @relation(fields: [pedidoId], references: [id], onDelete: Cascade)
  producto   Producto @relation(fields: [productoId], references: [id])
}

model PagoPedido {
  id       Int            @id @default(autoincrement())
  pedidoId Int
  fecha    DateTime
  tipo     TipoPagoPedido
  monto    Decimal        @db.Decimal(14, 2) // puede ser negativo (crédito)
  nota     String?
```

```
  pedido   Pedido         @relation(fields: [pedidoId], references: [id])
}

model GastoRecepcion {
  id         Int      @id @default(autoincrement())
  pedidoId   Int
  concepto   String   // flete, seguro, impuestos, agente, almacenaje...
  monto      Decimal  @db.Decimal(14, 2)
  aproximado Boolean  @default(false)
  pedido     Pedido   @relation(fields: [pedidoId], references: [id], onDelete: Cascade)
}

// ---------- Caja, cuentas de banco y gastos ----------
model CuentaBanco { // cuentas de banco en lempiras de Caja
  id            Int     @id @default(autoincrement())
  nombre        String
  saldoLempiras Decimal @default(0) @db.Decimal(14, 2)
  orden         Int     @default(1)
}

model CategoriaGasto {
  id                 Int               @id @default(autoincrement())
  nombre             String
  grupo              GrupoGasto
  tipo               TipoGasto
  presupuestoMensual Decimal?          @db.Decimal(14, 2) // variables en meses futuros
  montoFijo          Decimal?          @db.Decimal(14, 2)
  activaDesde        DateTime?
  activaHasta        DateTime?
  esSistema          Boolean           @default(false)    // Devoluciones/cambios
  fijosMes           GastoFijoMes[]
  montosVigentes     MontoFijoVigente[]
  movimientos        MovimientoGasto[]
}

model MontoFijoVigente { // cada cambio de un gasto fijo "en adelante"
  id          Int            @id @default(autoincrement())
  categoriaId Int
  desdeAnio   Int
  desdeMes    Int
  monto       Decimal        @db.Decimal(14, 2)
  creadoEn    DateTime       @default(now())
  categoria   CategoriaGasto @relation(fields: [categoriaId], references: [id], onDelete: Cascade)

  @@index([categoriaId, desdeAnio, desdeMes])
}

model GastoFijoMes { // monto fijo de un mes específico ("solo este mes" o fijado al cerrar)
  id          Int            @id @default(autoincrement())
  categoriaId Int
  anio        Int
  mes         Int
  monto       Decimal        @db.Decimal(14, 2)
  real        Boolean        @default(false)
  categoria   CategoriaGasto @relation(fields: [categoriaId], references: [id], onDelete: Cascade)

  @@unique([categoriaId, anio, mes])
}
```

```
model MovimientoGasto { // gastos variables
  id          Int            @id @default(autoincrement())
  categoriaId Int
  fecha       DateTime
  monto       Decimal        @db.Decimal(14, 2)
  nota        String?
  categoria   CategoriaGasto @relation(fields: [categoriaId], references: [id])
}

model SaldoCaja { // registro único (id = 1), se escribe a mano
  id              Int      @id @default(1)
  efectivo        Decimal  @default(0) @db.Decimal(14, 2)
  cheques         Decimal  @default(0) @db.Decimal(14, 2)
  bancosUsd       Decimal  @default(0) @db.Decimal(14, 2)
  bancosUsdValorL Decimal  @default(0) @db.Decimal(14, 2)
  actualizadoEn   DateTime @updatedAt
}

model Anio {
  anio                     Int       @id
  metaAnual                Decimal   @default(0) @db.Decimal(14, 2)
  porcentajeBrutoAnterior  Decimal?  @db.Decimal(8, 6)
  cerrado                  Boolean   @default(false)
  cerradoEn                DateTime?
  multiplicadoresIniciales Json?     // por categoría, solo arranque
}

model MesContable { // Cierre de mes: PE y meta se fijan el día 1 (y al cerrar el arranque)
  id       Int      @id @default(autoincrement())
  anio     Int
  mes      Int
  peMes    Decimal  @db.Decimal(14, 2)
  metaMes  Decimal  @db.Decimal(14, 2)
  fijadoEn DateTime @default(now())

  @@unique([anio, mes])
}

// ---------- Demanda y auditoría ----------
model VentaBase { // año base 2026, editable
  id         Int      @id @default(autoincrement())
  productoId Int
  anio       Int
  mes        Int
  unidades   Int
  producto   Producto @relation(fields: [productoId], references: [id])

  @@unique([productoId, anio, mes])
}

model MesSinStock { // se registra cuando el stock llega a 0 en el mes, o el día 1 si empieza en 0
  id         Int      @id @default(autoincrement())
  productoId Int
  anio       Int
  mes        Int
  producto   Producto @relation(fields: [productoId], references: [id])

  @@unique([productoId, anio, mes])
}
```

```
model PeriodoSinStock { // historial de fechas en cero
  id         Int       @id @default(autoincrement())
  productoId Int
  desde      DateTime
  hasta      DateTime?
  producto   Producto  @relation(fields: [productoId], references: [id])

  @@index([productoId, desde])
}

model Historial {
  id        Int      @id @default(autoincrement())
  entidad   String
  entidadId String
  campo     String
  antes     String?
  despues   String?
  motivo    String?
  fecha     DateTime @default(now())

  @@index([entidad, entidadId])
}
```

Reglas que el modelo debe respetar (Especificación v1.1)

- **Operaciones con stock en transacción (`prisma.$transaction`):** ingresar pedido, editar o eliminar factura, ajustes y recepción de pedido deben descontar o devolver lotes y escribir el historial de forma atómica.
- **Stock de un código** = suma de `quedan` de sus lotes. **Apartado** = unidades en proformas pendientes. **Disponible** = `stock - apartado` (se calculan, no se guardan). El POS y el Catálogo usan el disponible; la Calculadora usa el stock.
- **Mes sin stock (regla actualizada en Especificación funcional v1.1 del 28/10/2026):** un mes cuenta como “sin stock” para un código si **en cualquier momento de ese mes su stock llegó a 0** (si durante todo el mes tuvo al menos 1 unidad, el mes cuenta con stock). Se registra automáticamente en `MesSinStock(productoId, anio, mes)` cada vez que una venta, una línea de Cambio o un ajuste deja el stock en 0, y el día 1 de cada mes para todo código que empiece el mes con stock 0.
- **Estado “En Arranque”:** mientras no exista ninguna factura de POS (`Factura` con `cargaInicial = false`), el sistema muestra los campos de arranque (lote Inicial en Productos, `+ Factura inicial` y `corregir saldo` en Ventas, `% bruto` manual y multiplicadores iniciales en Caja). La primera venta ingresada los oculta para siempre y crea el `MesContable` (`Cierre de mes`) del mes en curso.
- Una factura de carga inicial no tiene líneas; su monto ya abonado se registra como un abono con medio `CARGA_INICIAL`.
- Nada con movimiento se borra: se inactiva (`activo = false`) o se marca (`eliminada = true`).
- Toda edición relevante (precios, meta, gastos pasados, facturas, ajustes, lote Inicial, corrección de saldo, cierre y reapertura de año) escribe en `Historial`.

- Los repartos (gastos por CBM, FOB real por línea) dejan el residuo en la última línea para cuadrar al centavo.

10. Decisiones confirmadas y pendientes

Decisiones confirmadas

Tema	Decisión
Servidor	Ubuntu 22.04 LTS o 24.04 LTS, con Nginx y PostgreSQL 15/16. Compatible con CatalogoHN.
Node.js	Node 20 LTS (o 22 LTS). Compatible con CatalogoHN y con el ERP.
Copia externa de respaldos	Google Drive con `rclone` (costo $0). Carpeta privada de Drive; configuración en la sección 8.
Backups de DigitalOcean	Desactivados por ahora (opcional; costo extra del 20%). El `pg_dump` diario con `rclone` cubre el requisito de respaldo diario fuera del servidor.
Moneda y cálculos	`decimal.js` en el backend y `Intl.NumberFormat("es-HN", { style: "currency", currency: "HNL" })` en pantalla. Evita errores de punto flotante y permite dejar el residuo en la última línea para cuadrar al centavo.
PDFs	Se generan en el backend (`@react-pdf/renderer`), porque el respaldo semanal por correo corre sin navegador abierto. Para el usuario, el backend entrega el buffer o stream de descarga.
Excel	`exceljs` (el paquete `xlsx` público está desactualizado y tiene vulnerabilidades reportadas).
ORM	Prisma.
Despliegue	`git pull` manual en el Droplet, con PM2.
Dominio	`app.kolben.catalogohn.com` (producción) y `test.app.kolben.catalogohn.com` (pruebas).
Mes sin stock (v1.1)	Según la Especificación v1.1 (28/10/2026): un mes cuenta como “sin stock” para un código si en cualquier momento de ese mes su stock llegó a 0. Se registra automáticamente en `MesSinStock` al quedar en 0 (venta, Cambio o ajuste) y el día 1 de cada mes si inicia en 0. Alimenta la Calculadora desde 2028 (2027 usa el año base 2026).
Datos de arranque	Javier los carga manualmente por pantalla cuando reciba el sistema (sección 8 de la spec). No se desarrolla importador; sí datos semilla para pruebas.
Facturas	Internas, sin formato fiscal del SAR.
Entregas y pruebas	Dos avances a Javier: uno a la mitad y otro al terminar el sistema, validados con los 18 ejemplos numéricos (sección 11 de la spec) y las 35 pruebas de entrega (sección 12 de la spec).
Memoria del Droplet	Uso actual cerca del 45%: hay margen para el ERP; PM2 con límite de 512 MB por proceso.

Pendientes

#	Punto	Propuesta actual
1	Registros DNS de los subdominios	Crear un registro A para `app.kolben.catalogohn.com` y otro para `test.app.kolben.catalogohn.com` hacia la IP del Droplet.
2	Datos de ejemplo de la spec	Los números de los mockups son ilustrativos; los 18 ejemplos de la sección 11 de la spec v1.1 son pruebas exactas de aceptación.
3	Cuenta de Google para el respaldo	Definir qué cuenta de Google Drive se usará y quién la administra.
4	Cuenta gratuita de Healthchecks.io	Para los avisos de respaldo; crear una verificación por script y poner el correo de Diego.
5	Repositorio en GitHub	Crear `kolben-erp`, ramas `main` y `develop`, y proteger `main`.

11. Plan de trabajo por etapas

El plan sigue las 13 pantallas de la especificación (Entrada + pantallas 0 a 12) y agrega una etapa inicial de cimientos y una final de cierre, para un total de **15 etapas**. El orden respeta las dependencias: primero lo que alimenta a lo demás (productos, clientes), luego las ventas, después las compras y al final los tableros que leen de todo.

Los mockups de cada pantalla están en el **Anexo A** (al final del documento), con la etapa en la que se construye cada una.

Cómo se trabaja cada etapa

- Se programa en una rama `feature/etapa-N-nombre` creada desde `develop`; al terminar, merge a `develop` y despliegue a **pruebas**.
- Se prueba en el entorno de pruebas contra los criterios de “Listo cuando” y las pruebas de las secciones 11 y 12 de la spec v1.1. Solo si pasa, se hace merge a `main` y se despliega a producción (sección 5).
- Cada etapa incluye su migración de Prisma, su API, su pantalla, y los eventos de la sección 6 de la spec que le correspondan.
- Durante el desarrollo se carga data de prueba; la carga real (arranque, sección 8 de la spec) se hace al final, en la Etapa 14.

Etapas

Etapa	Pantalla (spec)	Qué se entrega	Depende de	Listo cuando
0	Cimientos	Repositorio y ramas; backend y frontend base; Prisma conectado; Nginx, HTTPS, PM2 y DNS en pruebas y producción; flujo de	—	Una página de prueba abre en test y en producción por

Etapa	Pantalla (spec)	Qué se entrega	Depende de	Listo cuando
		despliegue; `/api/health`; respaldo diario de base de datos y `uploads/` con aviso si falla; datos semilla para pruebas (sin correos reales).		HTTPS; el respaldo se restaura en pruebas.
1	Entrada + Menú y Configuración (12)	Login con JWT en cookie HttpOnly, creación del usuario en el primer uso, salir; menú lateral fijo (ancho 1280 px); datos de la proforma, bancos autorizados (orden), ISV, respaldo (correo y día), cambio de contraseña, enviar prueba; script para restablecer la contraseña desde el servidor.	0	Se entra y se sale; la configuración se guarda y persiste; sin sesión no se accede a ninguna ruta.
2	Productos (8)	Categorías (con empaque 10/20/50) y marcas con logo y orden; listado por categoría y marca con filtros y buscador; alta y edición de código (foto, CBM, descripción multilínea); orden con flechas y por número; inactivar (solo con stock 0, sin apartado ni pedido activo) y eliminar solo sin movimiento; campos de arranque (stock, FOB, costo que crean el lote Inicial); fotos y logos se redimensionan al subirlos (`sharp`); historial.	1	El orden guardado se respeta en la lista; se crea un código con lote Inicial; el valor del inventario a costo y a precio de venta cuadra (11.11).
3	Ficha de código (9)	Detalle con stock, apartado, disponible, valores y costo de lo que sale ahora (lote más viejo); lotes FIFO (agotados de años anteriores agrupados); ajustes de stock en dos pasos con motivo; edición del lote Inicial con motivo y recálculo de piezas ya vendidas; servicio FIFO reutilizable; **registro automático en `MesSinStock`** cuando el stock llega a 0 (y apertura/cierre en `PeriodoSinStock`).	2	Un ajuste saca del lote más viejo y entra al más nuevo; dejar un código en 0 registra ese mes en `MesSinStock` (11.17); editar el lote Inicial recalcula líneas vendidas.
4	Clientes (6)	Zonas y ciudades; lista ordenada por U.B. del año, buscador, sección “Deberían pedir este mes” (ritmo = mediana con ≥3 compras, ámbar/rojo/sin comprar hace mucho), tabla por zona y ficha del cliente; alta, edición e inactivación; precios por cliente con historial y marca de “sin precio”.	1, 2	Ciudad asigna zona sola; un cliente con saldo no se inactiva; el historial de precios guarda cada cambio; ritmo y colores dan como en 11.16.
5	POS · Proformas (1)	Buscador de cliente (con estado de cuenta) y de códigos con disponible; líneas con precio de la lista, precio cambiado y línea Cambio; totales e ISV; %U.B. solo en pantalla (alerta <40%); guardar proforma (aparta unidades);	2, 3, 4	Aparta y libera unidades; el PDF coincide con el mockup; ingresar saca stock FIFO y guarda `LoteConsumo` con costo y FOB como en 11.2.

Etapa	Pantalla (spec)	Qué se entrega	Depende de	Listo cuando
		pendientes con alerta de 7 días; PDF al cliente con `@react-pdf/renderer`; ingresar pedido con confirmación en línea (crea factura y cierra el arranque en la primera venta).		
6	Ventas (7)	Por cobrar (zona, ciudad, cliente), todas las ventas y cambios; abonos con recibo y medio; cheque post-fechado (`PENDIENTE`); editar (bajar cantidades / rechazo parcial) y quitar factura con confirmación; PDF de factura; `+ Factura inicial` (abono `CARGA_INICIAL`) y `corregir saldo` en arranque; respaldo semanal de Por cobrar por correo.	5	Editar o eliminar devuelve las piezas a su lote y recalcula el mes de la factura como en 11.3 y 11.4; el PDF semanal llega al correo de prueba.
7	Ventas por código (2)	Tabla por categoría, marca y código con unidades Ene–Dic y total anual; año base 2026 editable con historial; desde 2027 sale de las facturas válidas (sin Cambios) y se marcan visualmente los meses en `MesSinStock`; selector de año.	6	Los totales coinciden con las facturas; el año base se edita y se guarda; un mes sin stock aparece marcado.
8	Utilidad por código (3)	Drill-down año → categoría → marca → código con unidades, ingreso, costo FIFO, utilidad y %U.B.; marcas ordenadas de mayor a menor utilidad.	6	La utilidad coincide con la suma de facturas válidas; Cambios y cargas iniciales no cuentan.
9	Compras · Pedidos (5)	Pedidos activos con estados; pagos al proveedor y gastos (monto negativo = crédito); “Proveedor pagado completo”; marcar recibido con gastos reales, CBM del packing list y costeo por código; creación de lotes; bloqueos y avisos (sección 4.7); historial y descargable Excel por pedido.	2, 3	Al recibir, el costo de los lotes cuadra al centavo con FOB real + gastos como en 11.14; bloquea si el CBM no coincide con el packing list.
10	Compras · Calculadora (4)	Stock extra y ciclo de pedidos (`ParametrosCompra`, tránsito fijo de 5 meses); simulación mes a mes por código; demanda con regla de mes sin stock (promedio de meses con stock del mismo código; sin datos = 0); calculado redondeado a empaque con mínimo 50 u; borrador (`BorradorCalculadora`); totales contra el disponible; PDF y Excel de cotización; “Ingresar orden nueva”.	7, 9	El cálculo da exactamente el resultado de 11.15; la orden creada aparece en Pedidos y sube la deuda en Caja.

Etapa	Pantalla (spec)	Qué se entrega	Depende de	Listo cuando
11	Caja (10)	Saldos manuales (`SaldoCaja` y `CuentaBanco`); deuda por pedido y disponible para compras; meta anual, % bruto y multiplicadores iniciales; flujo de 12 meses con gastos fijos (`MontoFijoVigente` / `GastoFijoMes`) y variables; `Devoluciones/cambios` automáticas (÷12); PE y meta mensual; cerrar y reabrir año; corrección de meses pasados con historial; respaldo semanal de Caja (CSV).	6, 9	Caja Kolben, disponible, PE, metas, Devoluciones/cambios y balance cuadran con 11.5–11.9 y 11.12; editar gasto fijo respeta “solo este mes / en adelante”.
12	Resumen (0)	Tablero de solo lectura: ventas, U.B., U.N.1, U.N.2 y balance; gráfica mensual con PE (`MesContable`), meta y etiqueta “✓ PE cubierto”; barras anuales; por cobrar, inventario, multiplicadores (11.10), prueba ácida, posición financiera (Valor Kolben hoy y futuro) y pedidos en camino; selector de año.	6, 9, 11	Cada cifra coincide con su pantalla de origen y con los ejemplos 11.1–11.18; al elegir un año pasado, inventario, por cobrar, posición y pedidos siguen mostrando lo de hoy.
13	Catálogo (11)	Vista previa en cuadrícula 3×3; portada subida y paleta en `Configuracion` ; PDF con 6 fichas por página según el orden de Listas y Productos; solo códigos activos con disponible ≥ 1.	2	El PDF respeta el orden, sin precios ni títulos; un código con disponible 0 desaparece y los demás suben.
14	Cierre, arranque y salida	Procesos automáticos en servidor (cierre de mes el día 1 a las 00:00 HN que fija PE/meta y registra en `MesSinStock` los códigos que inician el mes en 0; respaldo semanal); ejecución completa de las **35 pruebas de entrega (sección 12 de la spec)**; guía de carga inicial para Javier en producción (sección 8 de la spec); documento de entrega.	0–13	Las 35 pruebas de entrega pasan al 100%; el respaldo se restaura; Javier firma de conformidad.

Nota: al terminar las Etapas 6 y 9 ya se puede operar ventas y compras; las Etapas 7, 8, 12 y 13 son lectura y reportes, y pueden reordenarse según lo que Javier necesite primero. Los tiempos por etapa no están incluidos: se estiman una vez confirmado el orden. Los dos avances a Javier se explican en la sección 12.

12. Antes de comenzar y entregas

Lista de verificación previa

Punto	Estado
Memoria del Droplet	Listo: uso actual cerca del 45%. Vigilar con `pm2 monit` al generar el catálogo por primera vez.
Límite de memoria en PM2	Listo: 512 MB por proceso (sección 6).
Repositorio en GitHub, ramas `main` y `develop`, `main` protegida	Completado (Etapa 0).
DNS de `app.` y `test.app.`	Pendiente configurar registros A hacia el Droplet.
Cuenta de Google Drive y `rclone`	Pendiente configurar en el Droplet.
Cuenta gratuita de Healthchecks.io	Pendiente configurar en el Droplet.
Datos semilla para pruebas	Se hacen en la Etapa 0 y se amplían con cada etapa. Nunca con el correo real de Javier.
Restablecer contraseña	Script en el servidor, Etapa 1 (completado).
Facturas internas (no SAR)	Confirmado.

Carga inicial de datos (Sección 8 de la spec)

Javier carga todo manualmente por pantalla cuando se le entregue el sistema; no hay importador. Mientras no exista ninguna factura de POS, el sistema está “en arranque” y muestra los campos especiales; la primera venta los oculta para siempre y crea el registro `Cierre de mes` del mes en curso. Orden de carga según la sección 8 de la spec:

Paso	Qué se carga	Dónde
1	Usuario y Configuración (proforma, bancos autorizados, ISV, respaldo)	Entrada y Configuración
2	Categorías y marcas	Productos · Listas
3	Productos con stock inicial, FOB/u y costo/u (crea el lote Inicial)	Productos
4	Zonas, ciudades, clientes y precios	Clientes
5	Facturas por cobrar viejas (`+ Factura inicial` con abono `CARGA_INICIAL`)	Ventas
6	Ventas por código 2026 (año base)	Ventas por código
7	Pedidos abiertos	Calculadora y Pedidos

Paso	Qué se carga	Dónde
8	Saldos, cuentas de banco, meta anual, % bruto 2026 y multiplicadores iniciales	Caja
9	Primera venta (el número de la proforma sigue a la última factura real; cierra el arranque)	POS

Avances a Javier

Se le muestran dos avances en el entorno de pruebas, ligados a sus dos pagos. Propuesta de corte (se puede mover):

Avance	Cuándo	Qué puede ver y probar
Avance 1 (mitad)	Al terminar la Etapa 6	Entrar al sistema, configurar, cargar productos y clientes con sus precios, hacer proformas e ingresarlas como facturas, ver Por cobrar y registrar abonos, y ver cómo se mueve el inventario por lotes.
Avance 2 (final)	Al terminar la Etapa 14	El sistema completo validado con las 35 pruebas de entrega: compras y calculadora, caja, resumen, catálogo, respaldos y la carga inicial en producción.

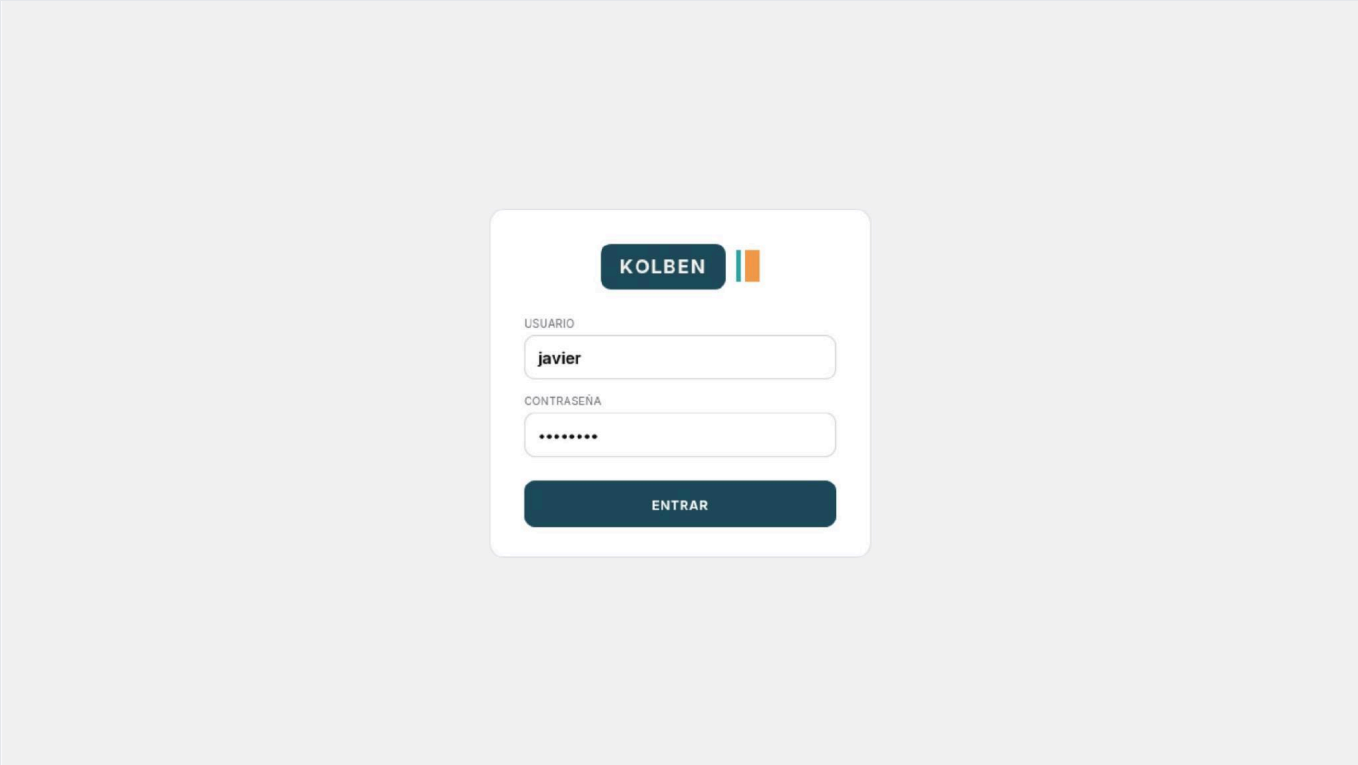
- Cada avance se pasa primero a pruebas (`test.app.kolben.catalogohn.com`) y Javier lo revisa ahí.
- Sus observaciones se anotan en una lista (por ejemplo en GitHub Issues o Projects) y se separan en correcciones de lo ya especificado y cambios nuevos fuera de la spec; estos últimos se acuerdan aparte antes de programarlos.
- Cuando Javier da su visto bueno, se pasa a `main` y a producción, con respaldo previo (sección 5).
- Se recomienda dejar por escrito qué incluye cada avance antes de presentarlo.

Anexo A. Mockups de la especificación

Mockups de alta fidelidad tomados de la Especificación funcional v1.1 (28/10/2026). Los números que muestran son de ejemplo (y validan los cálculos de la sección 11 de la spec); lo que manda son las reglas y fórmulas. Los bloques “Estado siguiente” muestran lo que aparece al tocar cada botón. Cada pantalla indica la etapa del plan (sección 11) en la que se construye.

A.1 Entrada (login) · Etapa 1

Pantalla de entrada



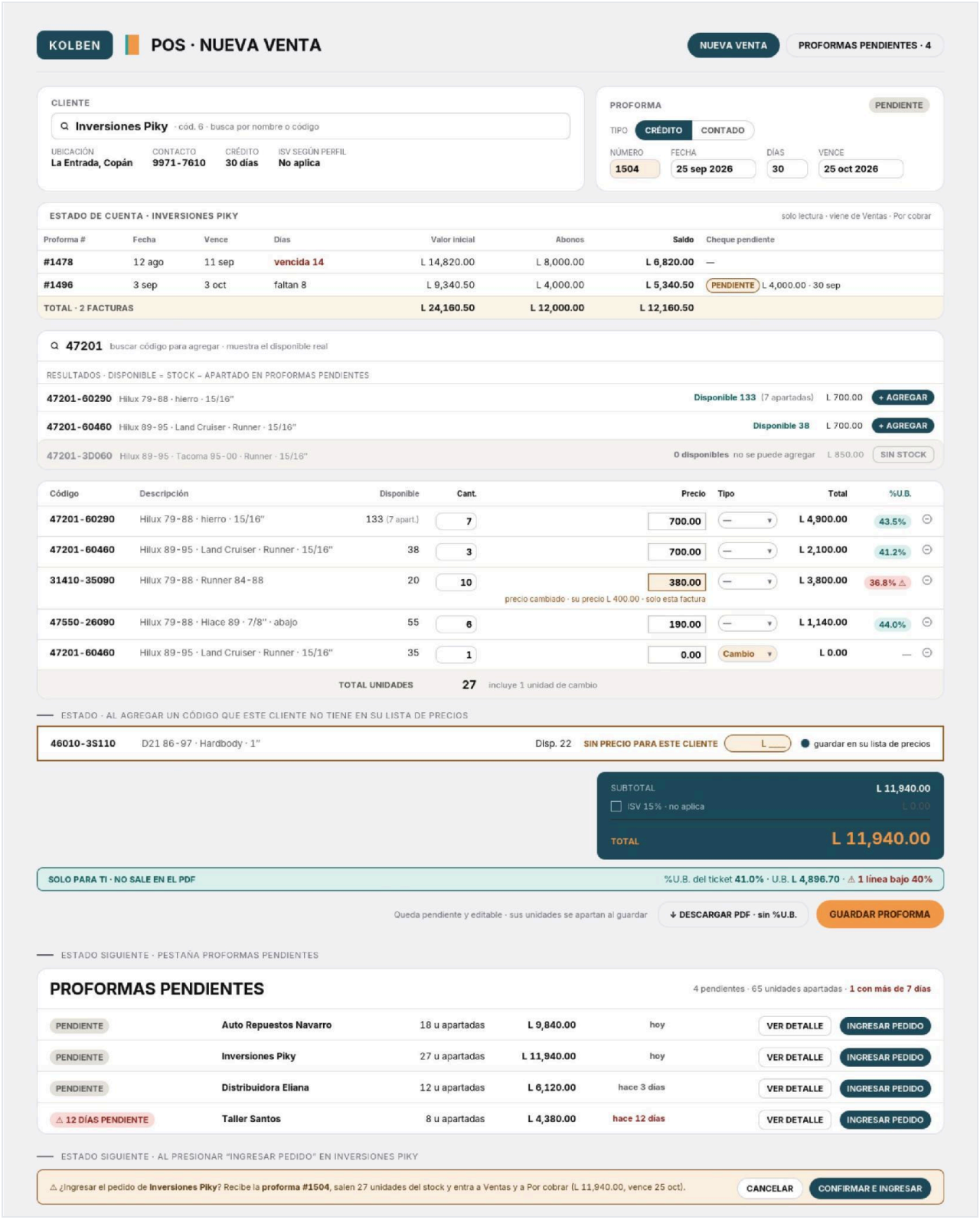
A.2 Pantalla 0 · Resumen · Etapa 12

Resumen

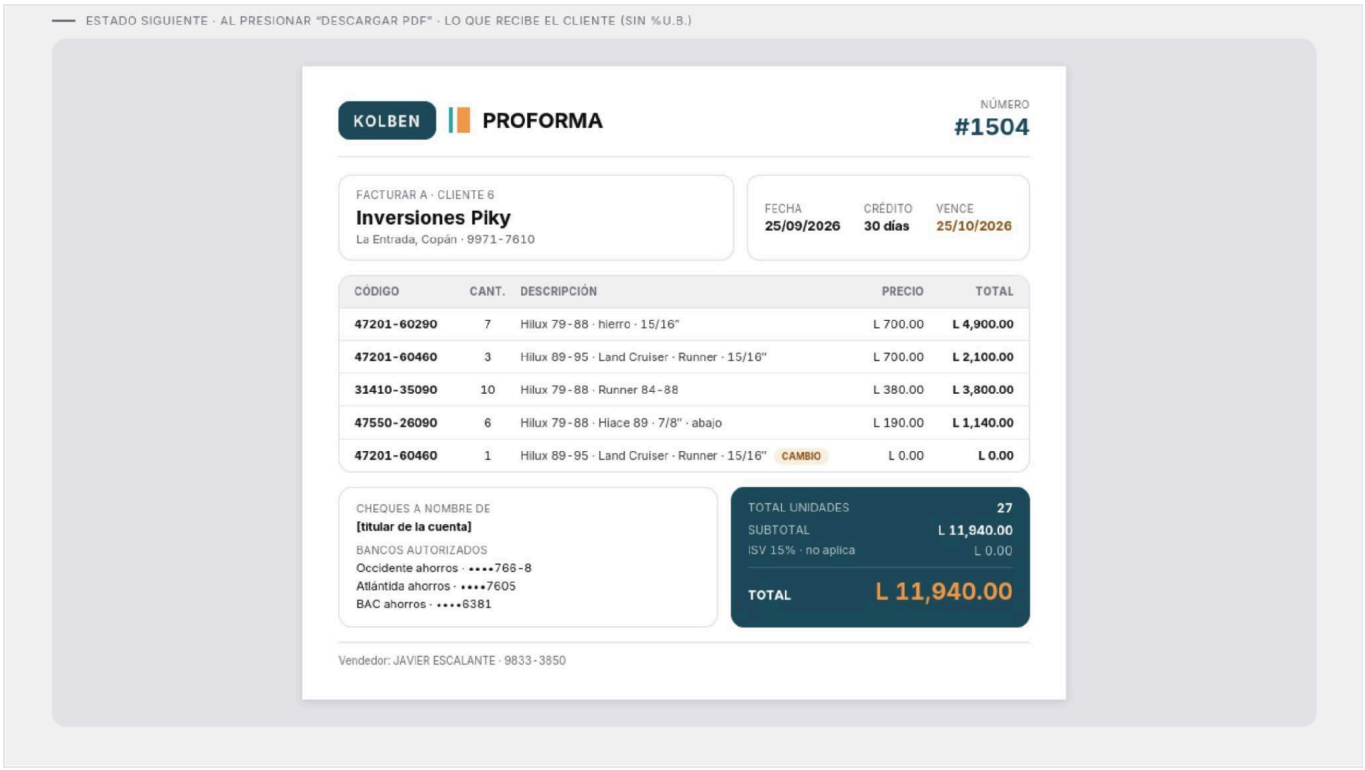


A.3 Pantalla 1 · POS · Etapa 5

POS y estados siguientes

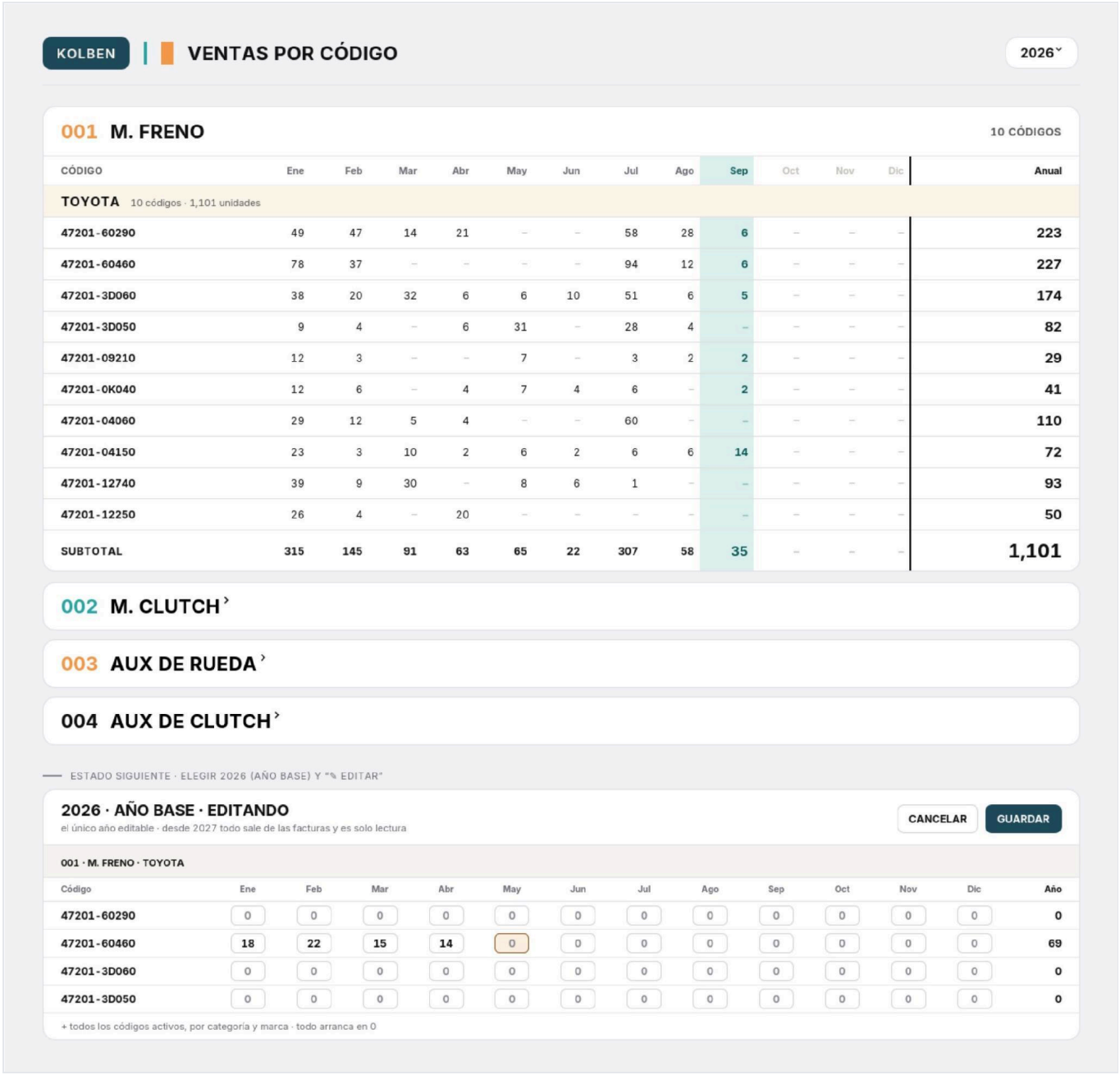


PDF de la proforma al cliente



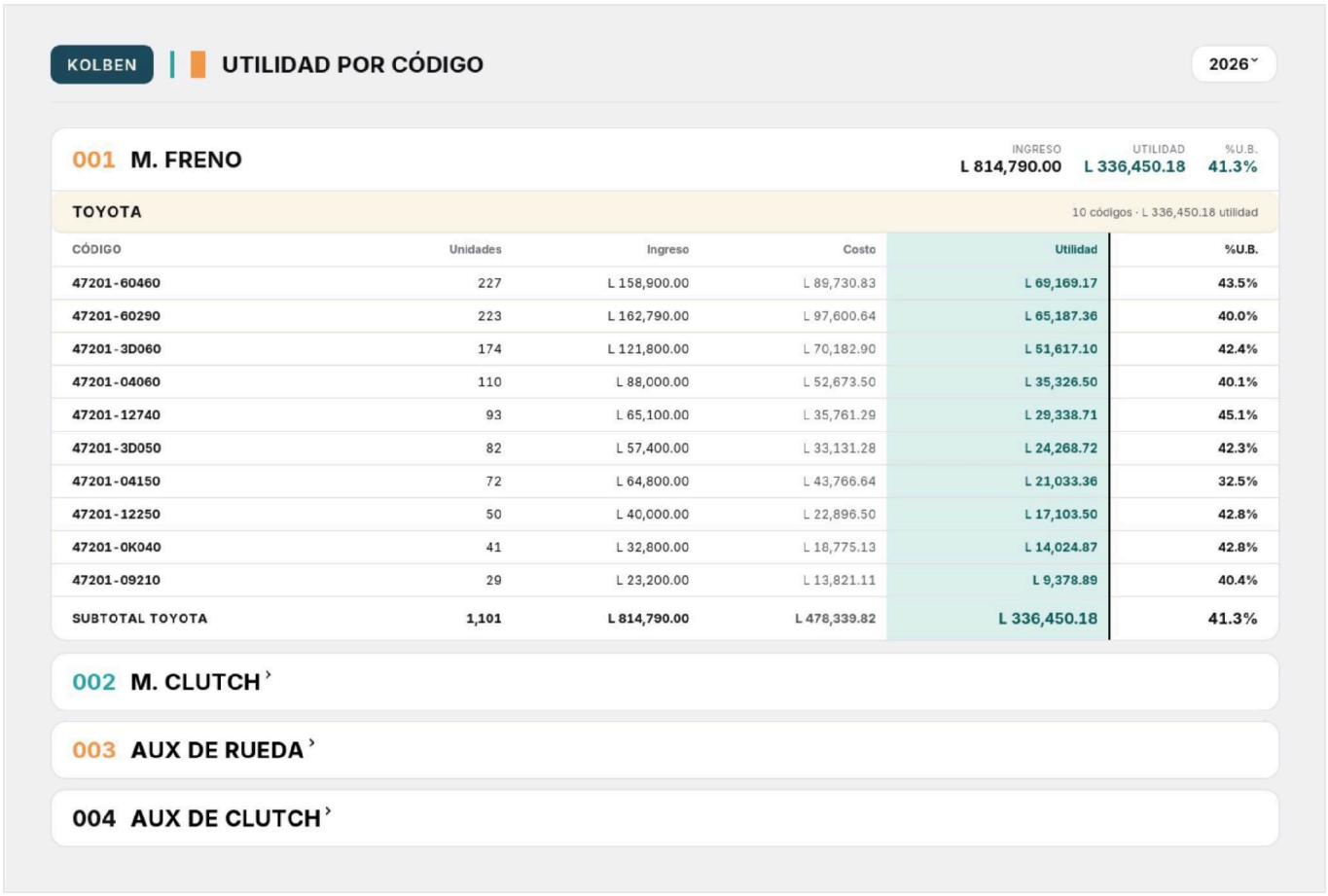
A.4 Pantalla 2 · Ventas por código · Etapa 7

Ventas por código y edición del año base



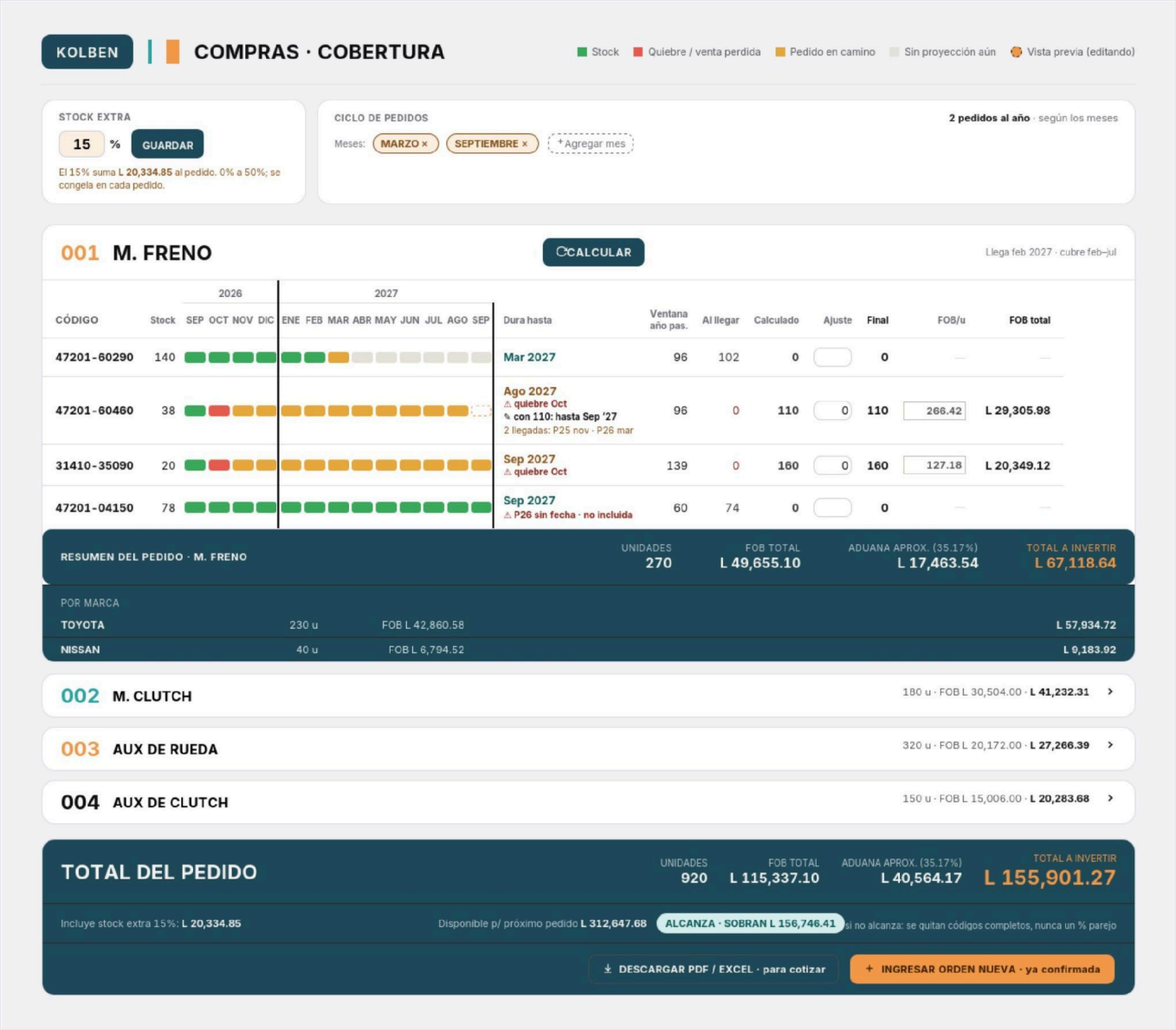
A.5 Pantalla 3 · Utilidad por código · Etapa 8

Utilidad por código



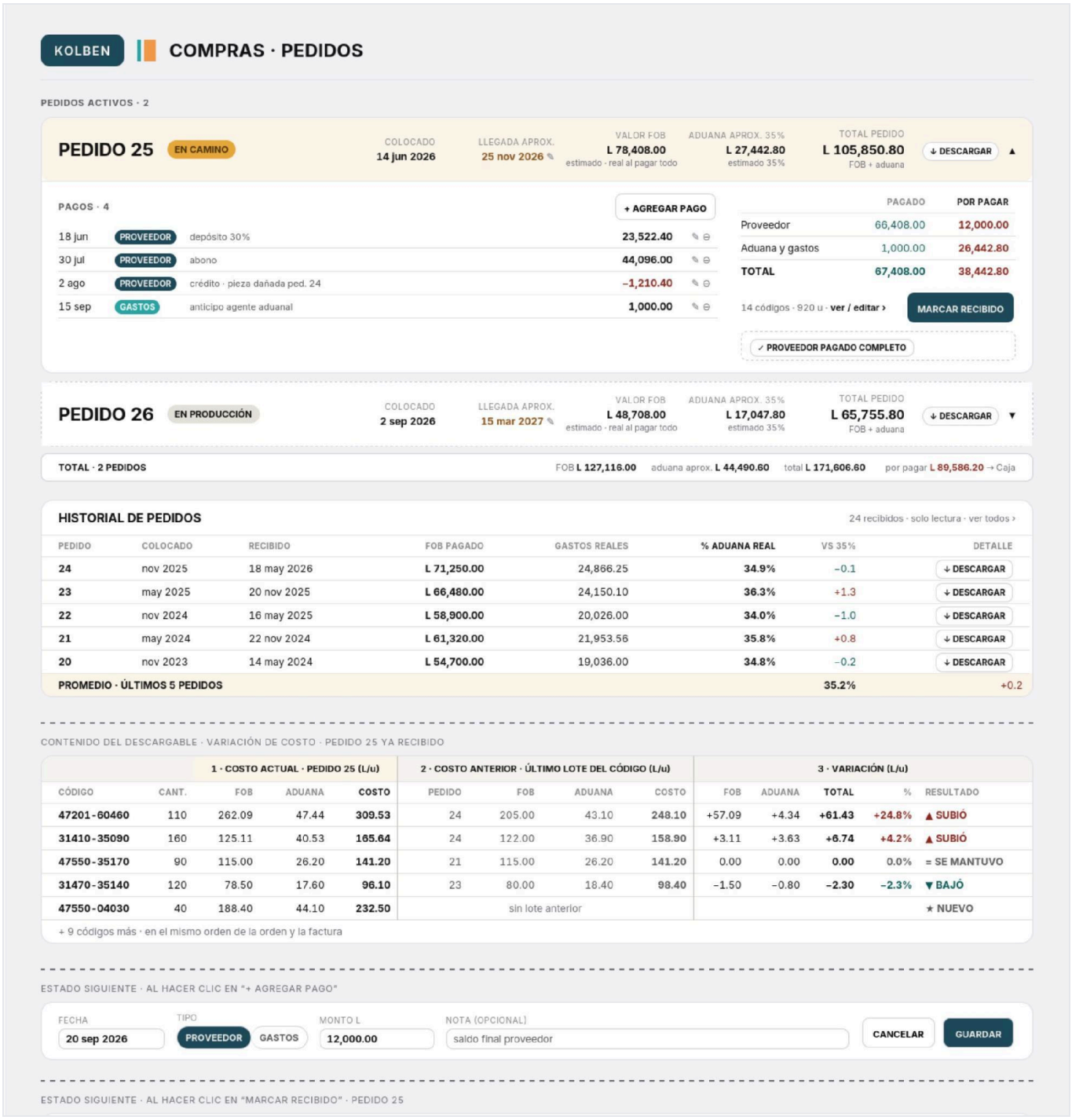
A.6 Pantalla 4 · Compras · Calculadora · Etapa 10

Calculadora de pedido



A.7 Pantalla 5 · Compras · Pedidos · Etapa 9

Pedidos, historial y recepción

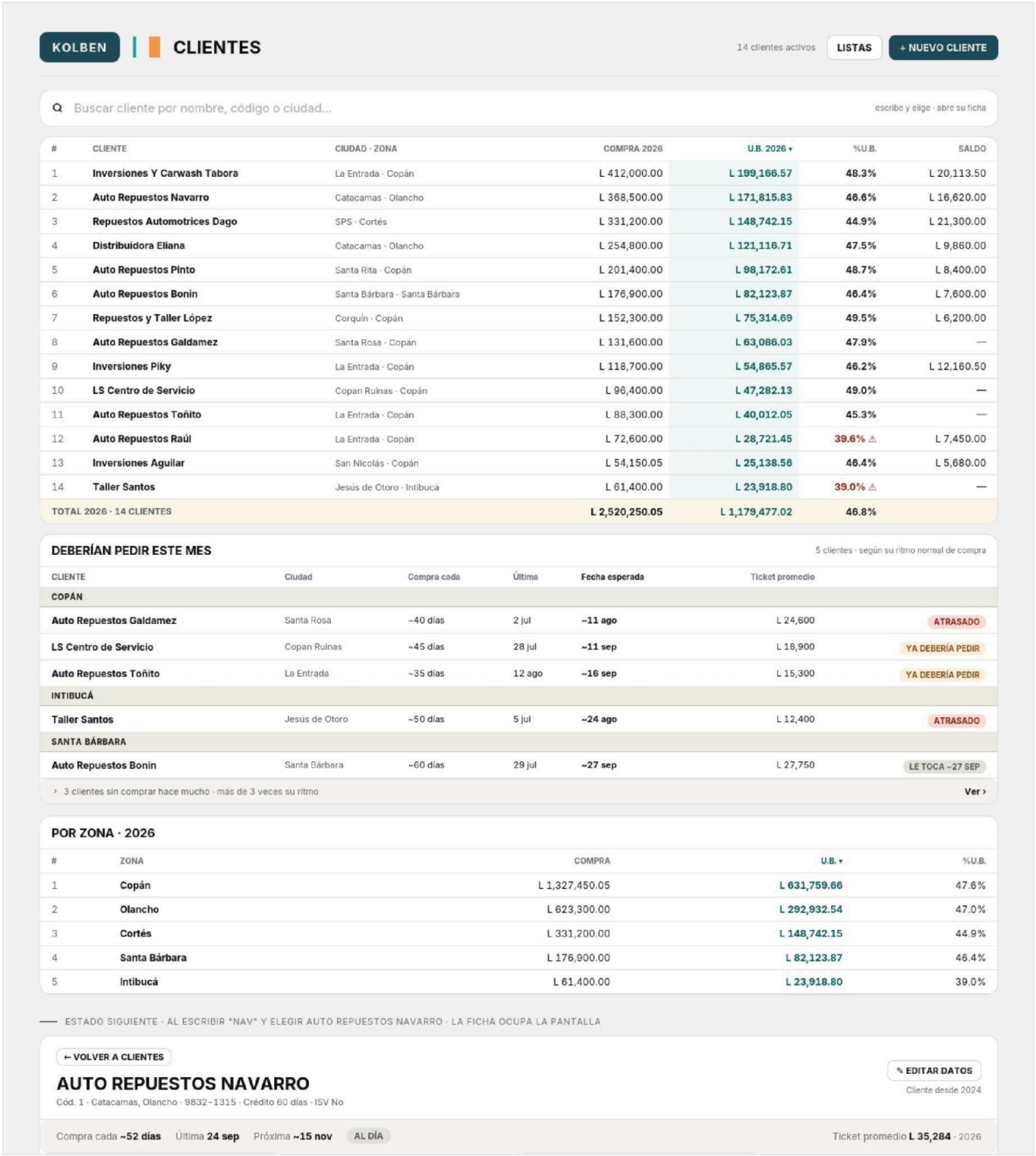


Recepción (continuación)

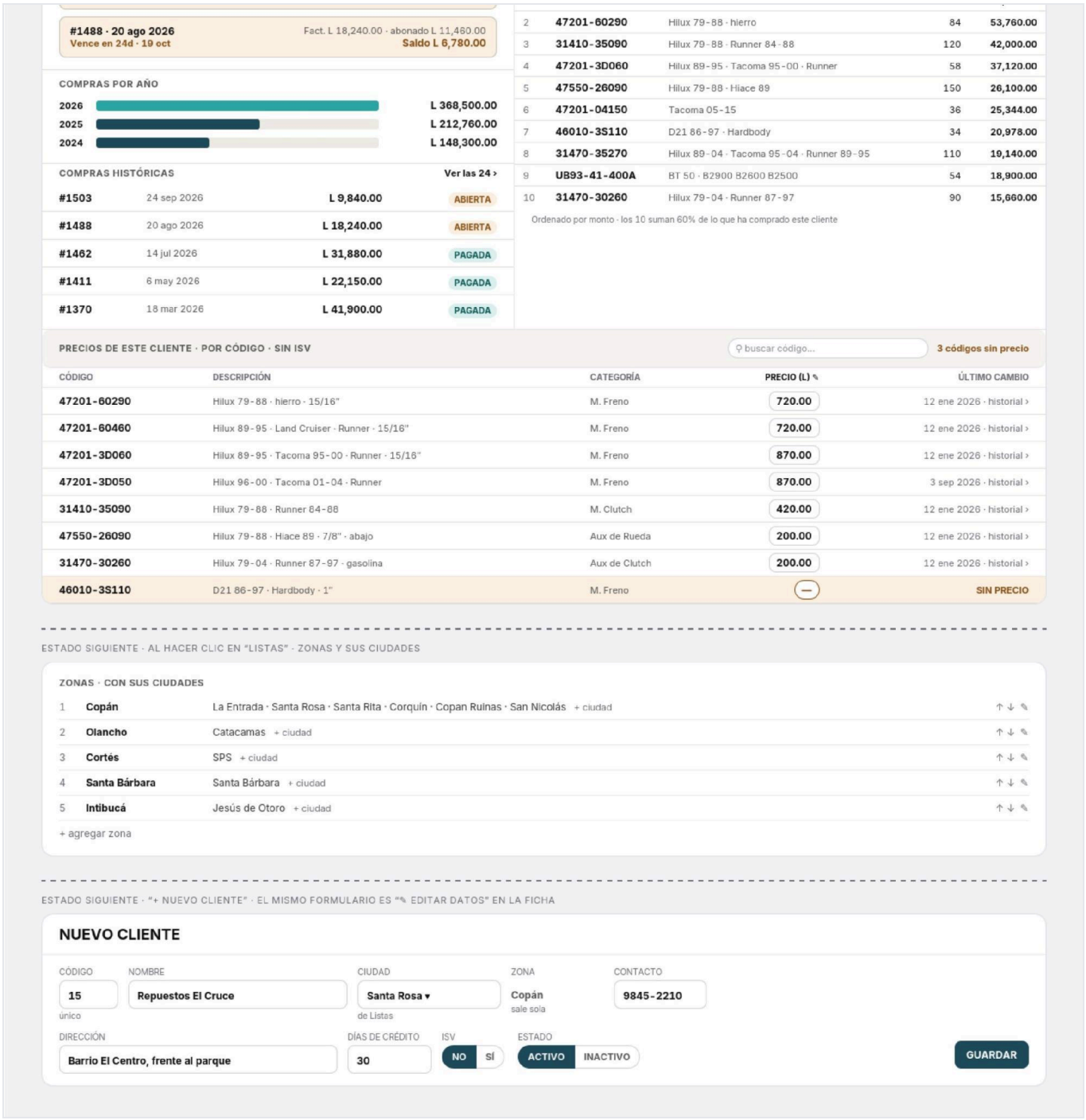


A.8 Pantalla 6 · Clientes · Etapa 4

Clientes y ficha

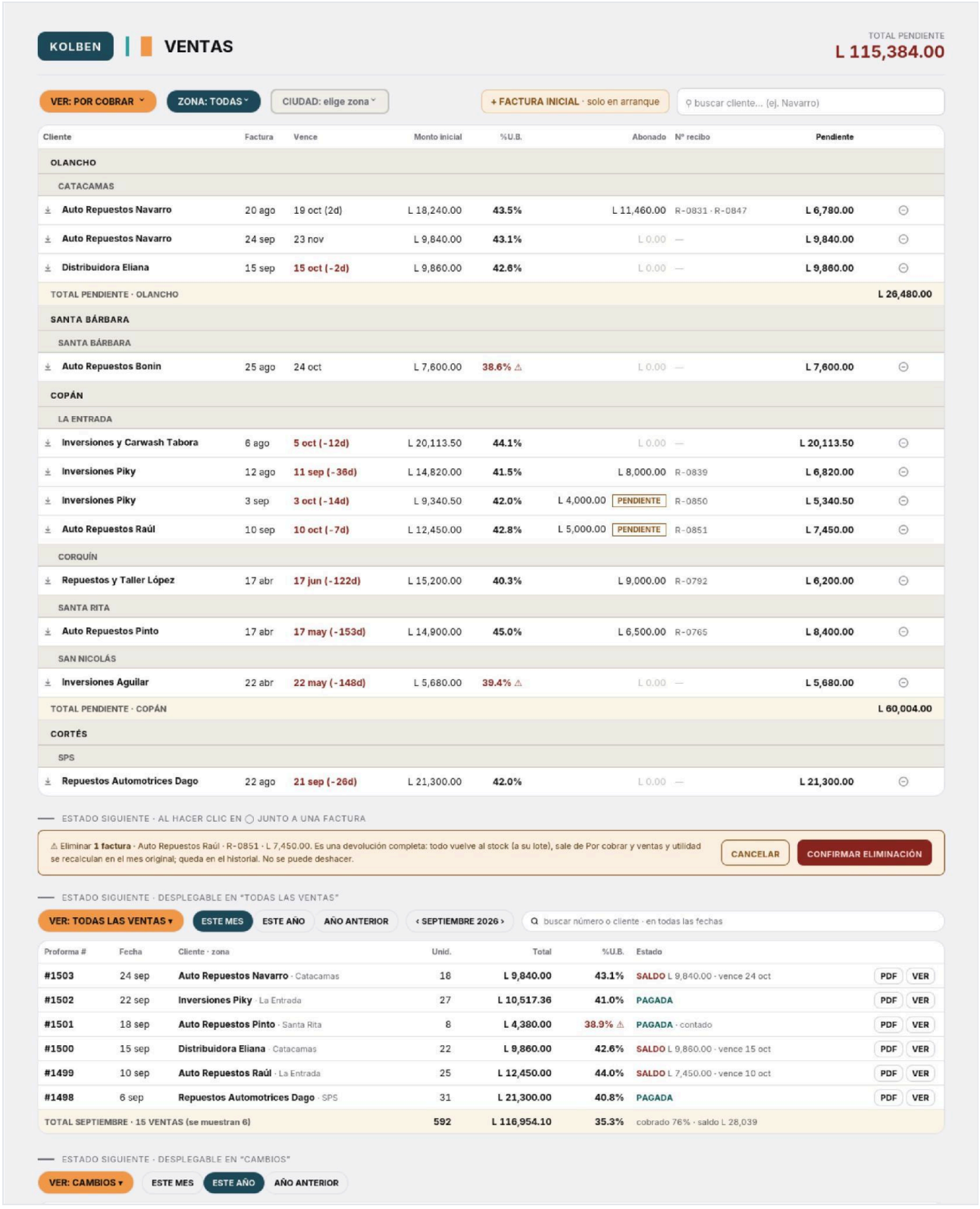


Ficha, precios, listas y nuevo cliente

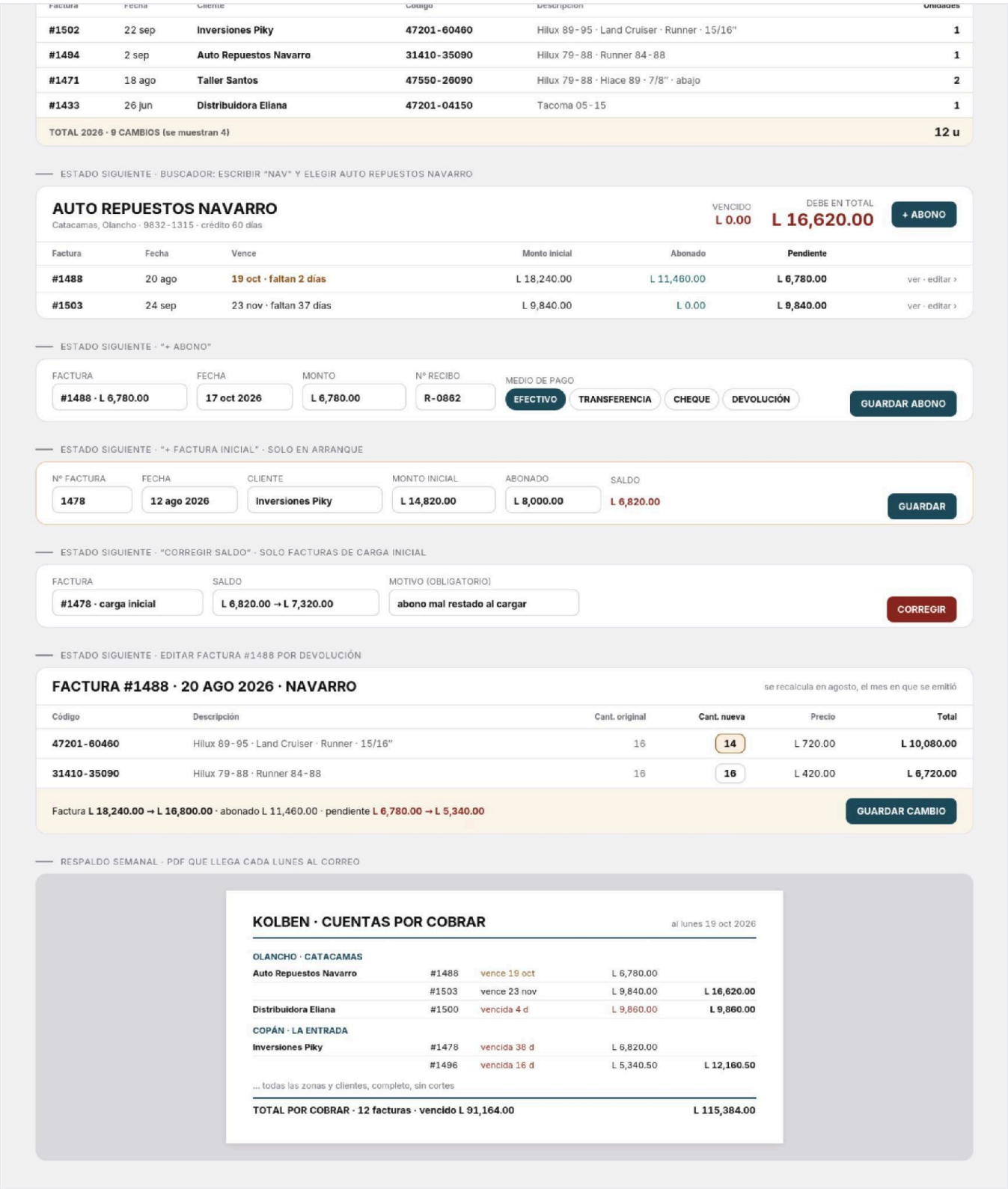


A.9 Pantalla 7 · Ventas · Etapa 6

Por cobrar, todas las ventas y cambios



Abonos, factura inicial, edición y respaldo semanal



Factura	Fecha	Cliente	Código	Descripción	Unidades
#1502	22 sep	Inversiones Piky	47201-60460	Hilux 89-95 · Land Cruiser · Runner · 15/16"	1
#1494	2 sep	Auto Repuestos Navarro	31410-35090	Hilux 79-88 · Runner 84-88	1
#1471	18 ago	Taller Santos	47550-26090	Hilux 79-88 · Hiace 89 · 7/8" · abajo	2
#1433	26 jun	Distribuidora Eliana	47201-04150	Tacoma 05-15	1
TOTAL 2026 · 9 CAMBIOS (se muestran 4)					12 u

ESTADO SIGUIENTE · BUSCADOR: ESCRIBIR "NAV" Y ELEGIR AUTO REPUESTOS NAVARRO

AUTO REPUESTOS NAVARRO
Catacamas, Olancho · 9832-1315 · crédito 60 días

VENCIDO L 0.00 · DEBE EN TOTAL L 16,620.00 · + ABONO

Factura	Fecha	Vence	Monto inicial	Abonado	Pendiente	
#1488	20 ago	19 oct · faltan 2 días	L 18,240.00	L 11,460.00	L 6,780.00	ver · editar ›
#1503	24 sep	23 nov · faltan 37 días	L 9,840.00	L 0.00	L 9,840.00	ver · editar ›

ESTADO SIGUIENTE · "+ ABONO"

FACTURA: #1488 · L 6,780.00 | FECHA: 17 oct 2026 | MONTO: L 6,780.00 | Nº RECIBO: R-0862 | MEDIO DE PAGO: EFECTIVO · TRANSFERENCIA · CHEQUE · DEVOLUCIÓN | GUARDAR ABONO

ESTADO SIGUIENTE · "+ FACTURA INICIAL" · SOLO EN ARRANQUE

Nº FACTURA: 1478 | FECHA: 12 ago 2026 | CLIENTE: Inversiones Piky | MONTO INICIAL: L 14,820.00 | ABONADO: L 8,000.00 | SALDO: L 6,820.00 | GUARDAR

ESTADO SIGUIENTE · "CORREGIR SALDO" · SOLO FACTURAS DE CARGA INICIAL

FACTURA: #1478 · carga inicial | SALDO: L 6,820.00 → L 7,320.00 | MOTIVO (OBLIGATORIO): abono mal restado al cargar | CORREGIR

ESTADO SIGUIENTE · EDITAR FACTURA #1488 POR DEVOLUCIÓN

FACTURA #1488 · 20 AGO 2026 · NAVARRO

se recalcula en agosto, el mes en que se emitió

Código	Descripción	Cant. original	Cant. nueva	Precio	Total
47201-60460	Hilux 89-95 · Land Cruiser · Runner · 15/16"	16	14	L 720.00	L 10,080.00
31410-35090	Hilux 79-88 · Runner 84-88	16	16	L 420.00	L 6,720.00

Factura L 18,240.00 → L 16,800.00 · abonado L 11,460.00 · pendiente L 6,780.00 → L 5,340.00 | GUARDAR CAMBIO

RESPALDO SEMANAL · PDF QUE LLEGA CADA LUNES AL CORREO

KOLBEN · CUENTAS POR COBRAR — al lunes 19 oct 2026

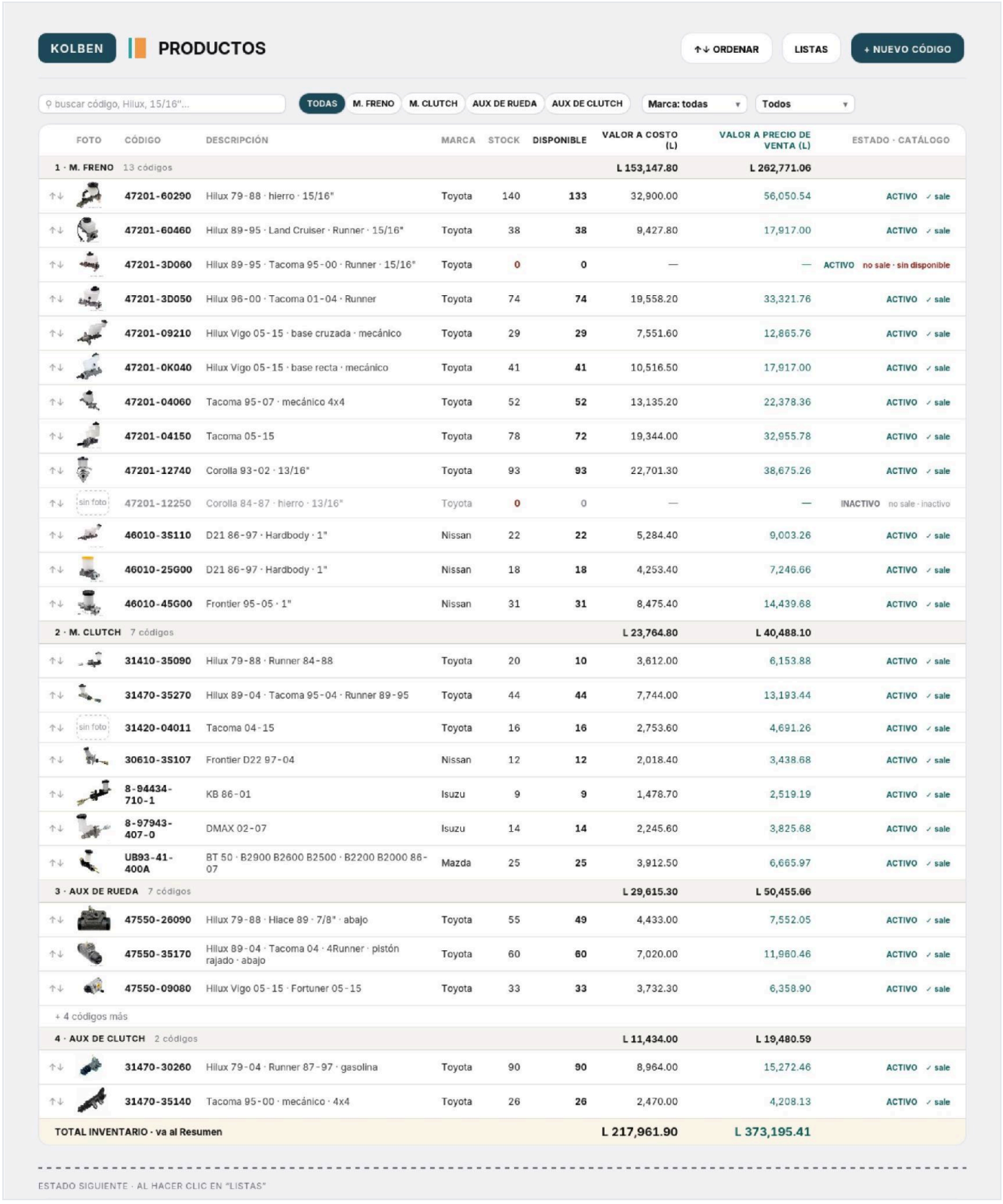
OLANCHO · CATACAMAS				
Auto Repuestos Navarro	#1488	vence 19 oct	L 6,780.00	
	#1503	vence 23 nov	L 9,840.00	L 16,620.00
Distribuidora Eliana	#1500	vencida 4 d	L 9,860.00	L 9,860.00
COPÁN · LA ENTRADA				
Inversiones Piky	#1478	vencida 38 d	L 6,820.00	
	#1496	vencida 16 d	L 5,340.50	L 12,160.50

... todas las zonas y clientes, completo, sin cortes

TOTAL POR COBRAR · 12 facturas · vencido L 91,164.00 — L 115,384.00

A.10 Pantalla 8 · Productos · Etapa 2

Productos y listas



KOLBEN

PRODUCTOS

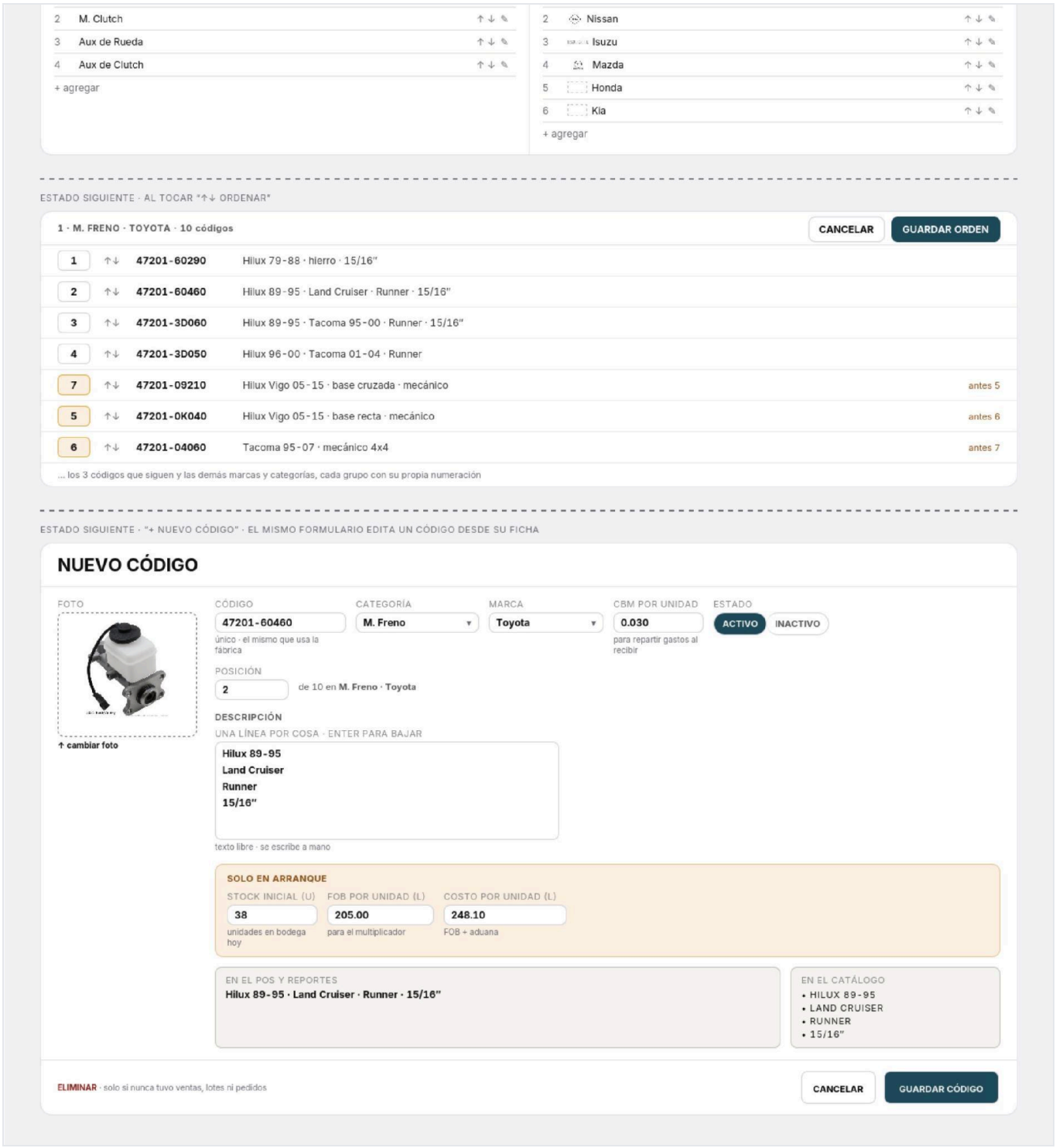
↑↓ ORDENAR | LISTAS | + NUEVO CÓDIGO

9 buscar código, Hilux, 15/16"... | TODAS | M. FRENO | M. CLUTCH | AUX DE RUEDA | AUX DE CLUTCH | Marca: todas | Todos

	FOTO	CÓDIGO	DESCRIPCIÓN	MARCA	STOCK	DISPONIBLE	VALOR A COSTO (L)	VALOR A PRECIO DE VENTA (L)	ESTADO · CATÁLOGO
1 · M. FRENO 13 códigos							**L 153,147.80**	**L 262,771.06**	
↑↓		**47201-60290**	Hilux 79-88 · hierro · 15/16"	Toyota	140	**133**	32,900.00	56,050.54	ACTIVO ✓ sale
↑↓		**47201-60460**	Hilux 89-95 · Land Cruiser · Runner · 15/16"	Toyota	38	**38**	9,427.80	17,917.00	ACTIVO ✓ sale
↑↓		**47201-3D060**	Hilux 89-95 · Tacoma 95-00 · Runner · 15/16"	Toyota	0	**0**	—	—	ACTIVO no sale · sin disponible
↑↓		**47201-3D050**	Hilux 96-00 · Tacoma 01-04 · Runner	Toyota	74	**74**	19,558.20	33,321.76	ACTIVO ✓ sale
↑↓		**47201-09210**	Hilux Vigo 05-15 · base cruzada · mecánico	Toyota	29	**29**	7,551.60	12,865.76	ACTIVO ✓ sale
↑↓		**47201-0K040**	Hilux Vigo 05-15 · base recta · mecánico	Toyota	41	**41**	10,516.50	17,917.00	ACTIVO ✓ sale
↑↓		**47201-04060**	Tacoma 95-07 · mecánico 4x4	Toyota	52	**52**	13,135.20	22,378.36	ACTIVO ✓ sale
↑↓		**47201-04150**	Tacoma 05-15	Toyota	78	**72**	19,344.00	32,955.78	ACTIVO ✓ sale
↑↓		**47201-12740**	Corolla 93-02 · 13/16"	Toyota	93	**93**	22,701.30	38,675.26	ACTIVO ✓ sale
↑↓	sin foto	47201-12250	Corolla 84-87 · hierro · 13/16"	Toyota	0	0	—	—	INACTIVO no sale · inactivo
↑↓		**46010-3S110**	D21 86-97 · Hardbody · 1"	Nissan	22	**22**	5,284.40	9,003.26	ACTIVO ✓ sale
↑↓		**46010-25G00**	D21 86-97 · Hardbody · 1"	Nissan	18	**18**	4,253.40	7,246.66	ACTIVO ✓ sale
↑↓		**46010-45G00**	Frontier 95-05 · 1"	Nissan	31	**31**	8,475.40	14,439.68	ACTIVO ✓ sale
2 · M. CLUTCH 7 códigos							**L 23,764.80**	**L 40,488.10**	
↑↓		**31410-35090**	Hilux 79-88 · Runner 84-88	Toyota	20	**10**	3,612.00	6,153.88	ACTIVO ✓ sale
↑↓		**31470-35270**	Hilux 89-04 · Tacoma 95-04 · Runner 89-95	Toyota	44	**44**	7,744.00	13,193.44	ACTIVO ✓ sale
↑↓	sin foto	**31420-04011**	Tacoma 04-15	Toyota	16	**16**	2,753.60	4,691.26	ACTIVO ✓ sale
↑↓		**30610-3S107**	Frontier D22 97-04	Nissan	12	**12**	2,018.40	3,438.68	ACTIVO ✓ sale
↑↓		**8-94434-710-1**	KB 86-01	Isuzu	9	**9**	1,478.70	2,519.19	ACTIVO ✓ sale
↑↓		**8-97943-407-0**	DMAX 02-07	Isuzu	14	**14**	2,245.60	3,825.68	ACTIVO ✓ sale
↑↓		**UB93-41-400A**	BT 50 · B2900 B2600 B2500 · B2200 B2000 86-07	Mazda	25	**25**	3,912.50	6,665.97	ACTIVO ✓ sale
3 · AUX DE RUEDA 7 códigos							**L 29,615.30**	**L 50,455.66**	
↑↓		**47550-26090**	Hilux 79-88 · Hiace 89 · 7/8" · abajo	Toyota	55	**49**	4,433.00	7,552.05	ACTIVO ✓ sale
↑↓		**47550-35170**	Hilux 89-04 · Tacoma 04 · 4Runner · pistón rajado · abajo	Toyota	60	**60**	7,020.00	11,960.46	ACTIVO ✓ sale
↑↓		**47550-09080**	Hilux Vigo 05-15 · Fortuner 05-15	Toyota	33	**33**	3,732.30	6,358.90	ACTIVO ✓ sale
+ 4 códigos más									
4 · AUX DE CLUTCH 2 códigos							**L 11,434.00**	**L 19,480.59**	
↑↓		**31470-30260**	Hilux 79-04 · Runner 87-97 · gasolina	Toyota	90	**90**	8,964.00	15,272.46	ACTIVO ✓ sale
↑↓		**31470-35140**	Tacoma 95-00 · mecánico · 4x4	Toyota	26	**26**	2,470.00	4,208.13	ACTIVO ✓ sale
TOTAL INVENTARIO · va al Resumen							**L 217,961.90**	**L 373,195.41**	

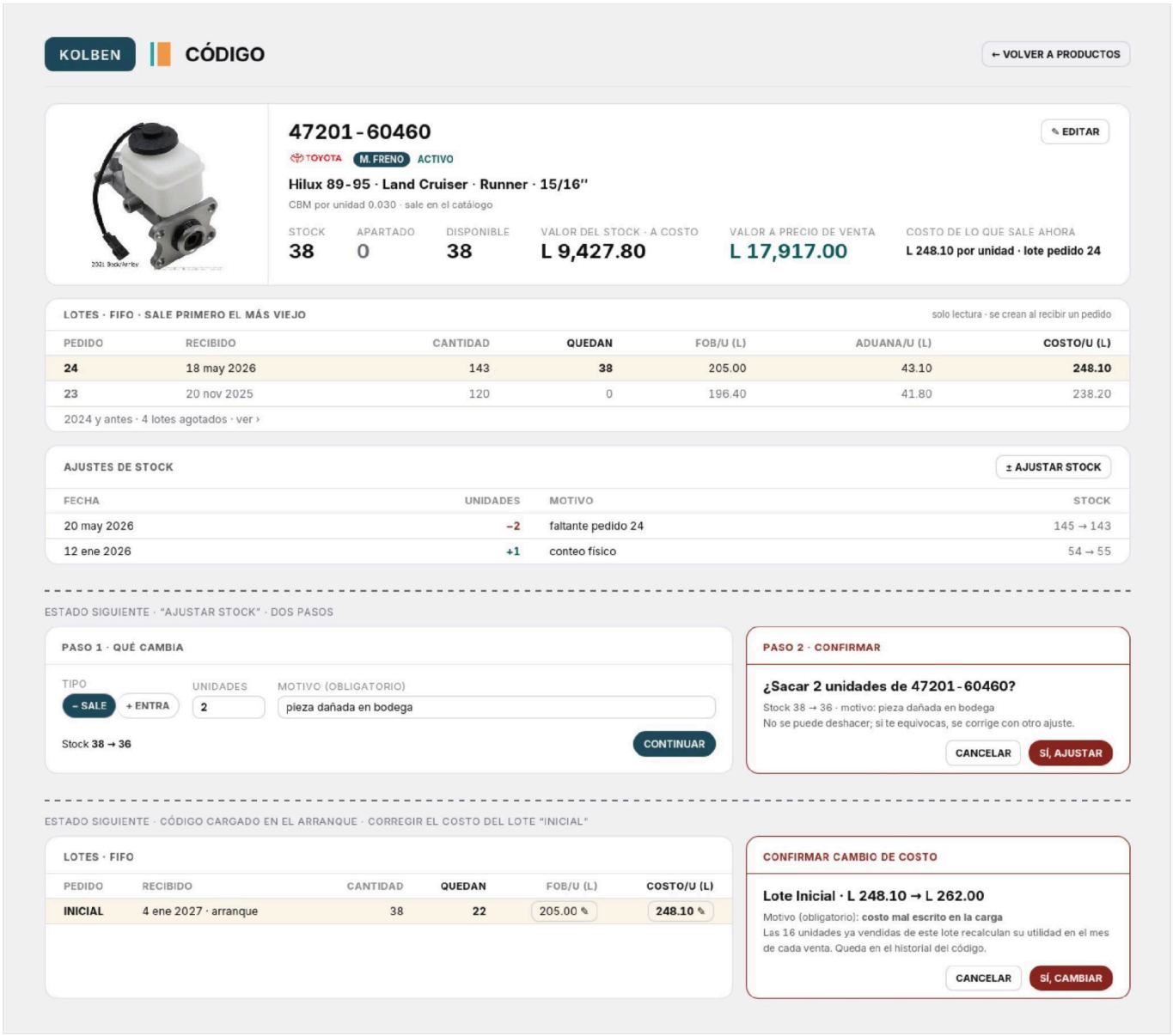
ESTADO SIGUIENTE · AL HACER CLIC EN "LISTAS"

Orden y nuevo código



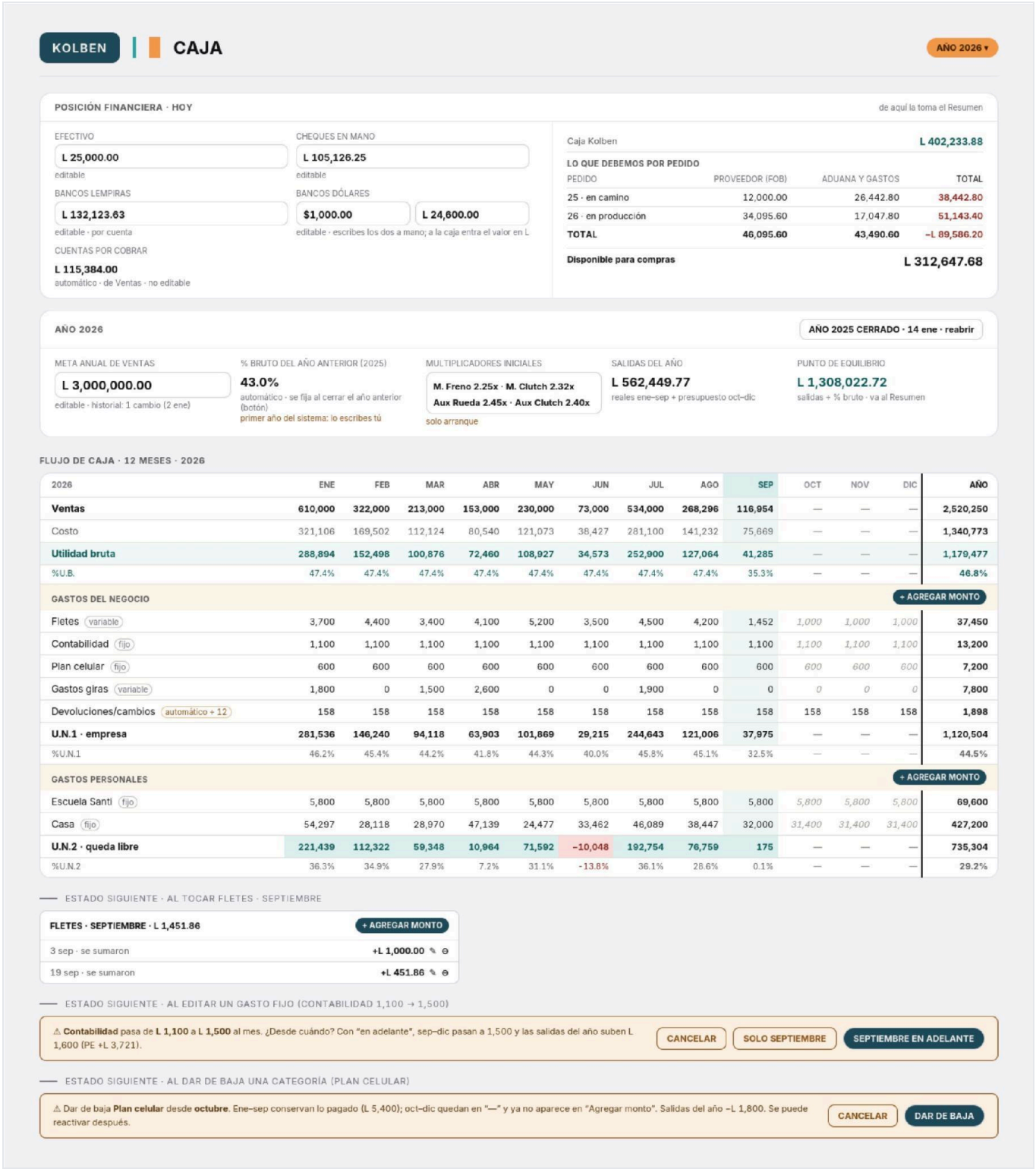
A.11 Pantalla 9 · Ficha de código · Etapa 3

Ficha de código, lotes y ajustes



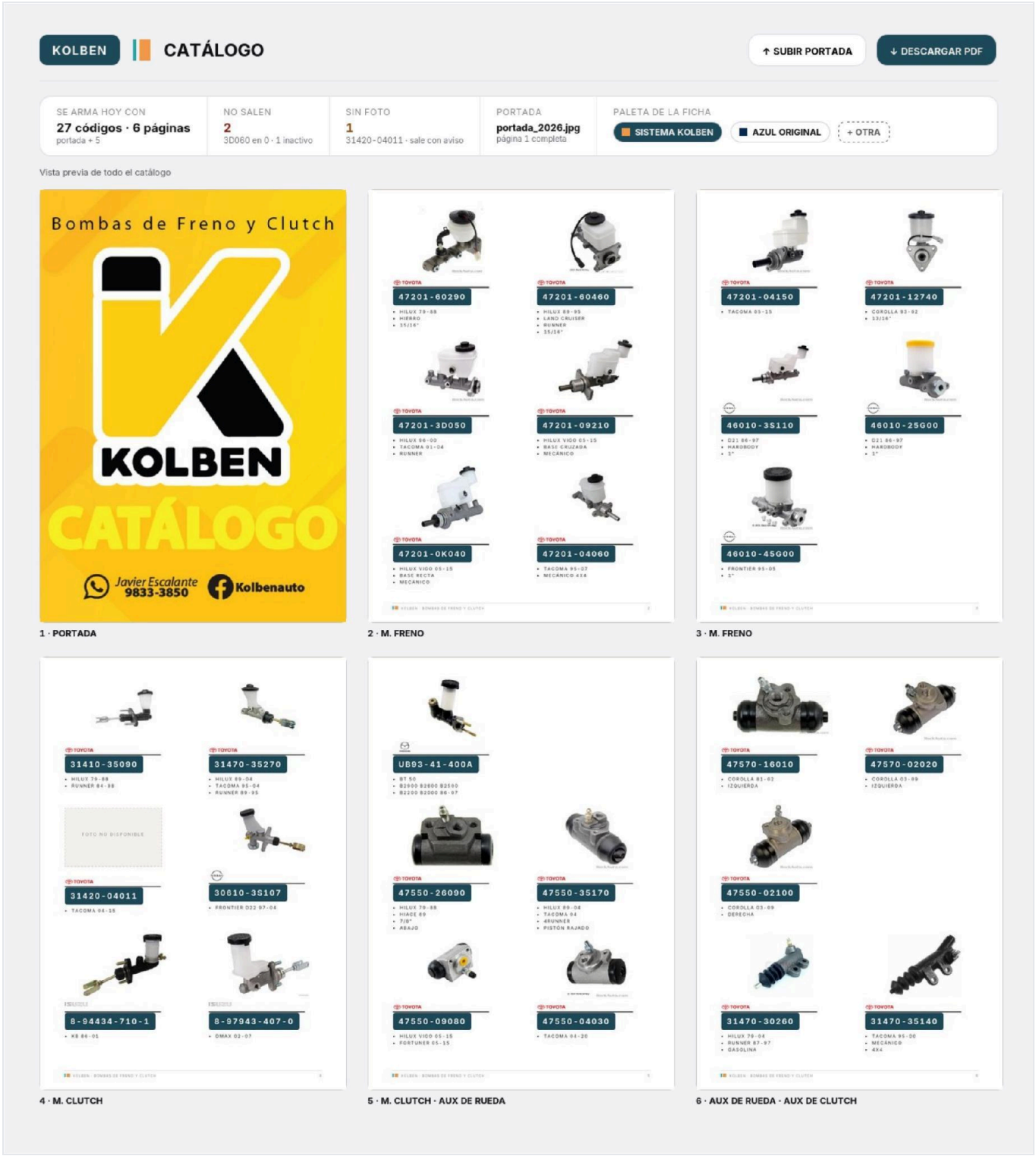
A.12 Pantalla 10 · Caja · Etapa 11

Caja y flujo de 12 meses



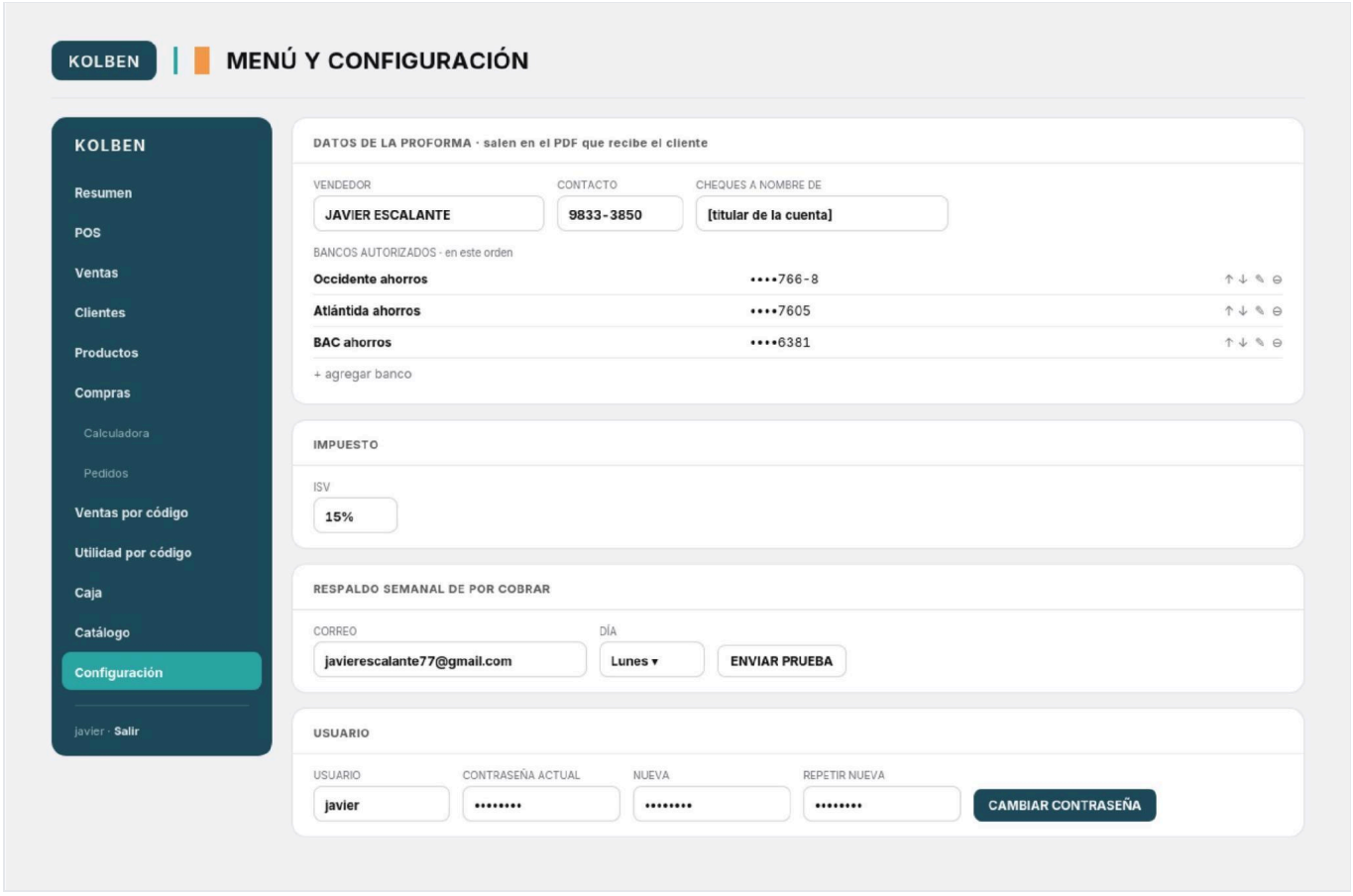
A.13 Pantalla 11 · Catálogo · Etapa 13

Catálogo PDF



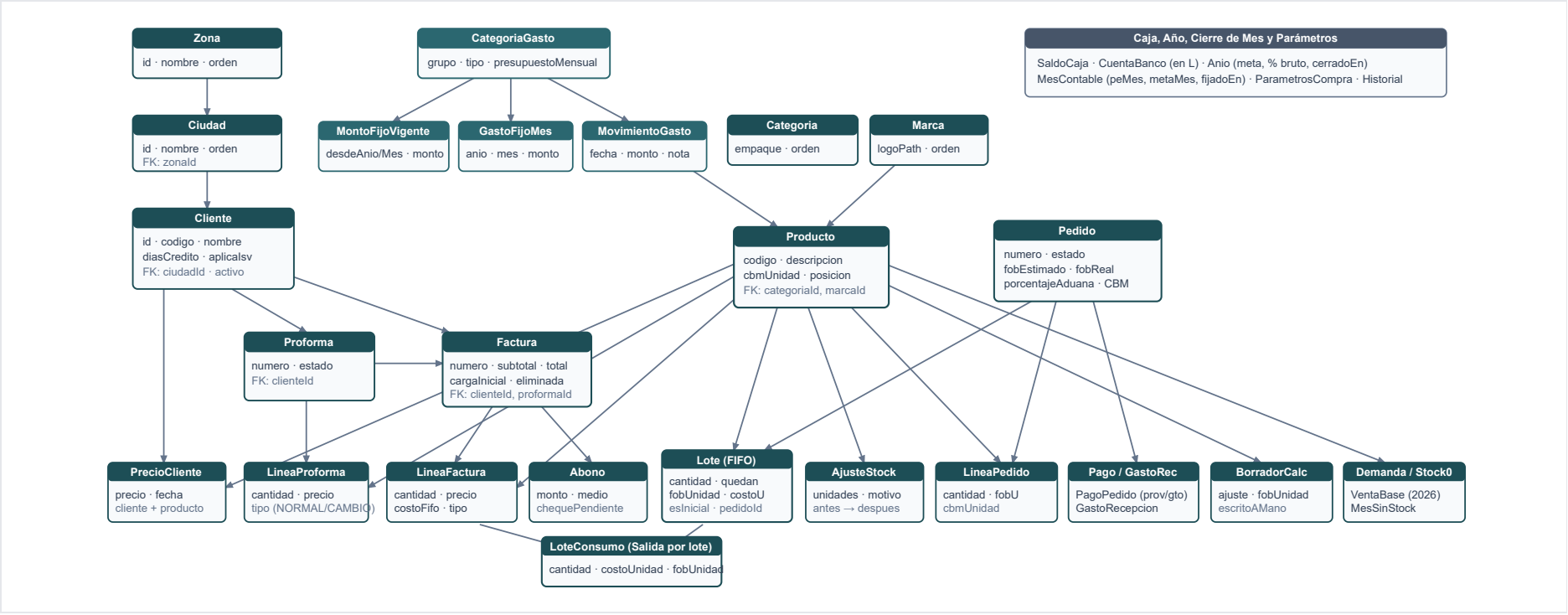
A.14 Pantalla 12 · Menú y Configuración · Etapa 1

Menú y Configuración



Anexo B. Diagrama de relaciones del modelo de datos

Tablas principales y sus relaciones actualizadas a la **Especificación funcional v1.1 (28/10/2026)**. No muestra todas las columnas (están en el esquema Prisma de la sección 9) ni las tablas independientes (`Usuario` , `Configuracion` , `Banco` , `ParametrosCompra` , `Historial`). Cada línea indica “uno a muchos”.



Bloques del modelo

- **Catálogo y clientes:** `Producto` (agrupado por `Categoria` con su `empaque` y `Marca`), `Cliente` (en una `Ciudad` , dentro de una `Zona`) y `PrecioCliente` .
- **Ventas:** `Proforma` y sus líneas; al ingresar el pedido se crea la `Factura` con sus `LineaFactura` y `Abono` (incluyendo medio `CARGA_INICIAL` en arranque).
- **Inventario FIFO:** el stock es la suma de `quedan` de los `Lote` ; `LoteConsumo` (Salida por lote) guarda de qué lote salió cada pieza con su `costoUnidad` y `fobUnidad` ; `AjusteStock` guarda las correcciones; `MesSinStock` registra automáticamente cada mes en que el stock de un código tocó 0 o empezó en 0.
- **Compras y Calculadora:** `ParametrosCompra` y `BorradorCalculadora` alimentan la simulación; `Pedido` con líneas, pagos y gastos de recepción crea los `Lote` con el costo real al recibirse.
- **Caja y reportes:** `SaldoCaja` , `CuentaBanco` (en L), `CategoriaGasto` con `MontoFijoVigente` , `GastoFijoMes` y `MovimientoGasto` , `Anio` , `MesContable` (`Cierre de mes`), `VentaBase` e `Historial` .